

# Module 6: Pandas DataFrame II

**Author:** Lei Wu

**Date:** 2023-01-01

**Version:** 1.0.0

## Learning Objectives

- Advanced DataFrame operations
- Grouping and aggregation
- Merging and joining DataFrames
- Pivot tables and reshaping
- Time series operations

## Topics Covered

1. GroupBy operations and aggregation
2. Pivot tables and cross-tabulation
3. Merging and joining DataFrames
4. Concatenating DataFrames
5. Reshaping data (melt, pivot, stack, unstack)
6. Time series operations
7. Window functions and rolling operations
8. Advanced filtering and querying
9. Multi-level indexing
10. Performance optimization techniques
11. Exporting and importing data

## Exercises

- Advanced data manipulation exercises
- Real-world data analysis projects

## 1. GroupBy Operations and Aggregation

In [ ...

```
1      import pandas as pd
2      import numpy as np
3
4
5      # Create a comprehensive dataset for groupby operations
6      np.random.seed(42)
7      data = {
8          'Department': ['IT', 'HR', 'Finance', 'IT', 'HR', 'Finance',
9          'Employee': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Fran
```

```

10         'Salary': np.random.normal(60000, 15000, 100),
11         'Experience': np.random.randint(1, 20, 100),
12         'Performance': np.random.choice(['Excellent', 'Good', 'Average', 'Poor'], 100),
13         'Location': np.random.choice(['New York', 'Los Angeles', 'Chicago'], 100)
14     }
15
16 df = pd.DataFrame(data)
17 df['Salary'] = df['Salary'].round(0).astype(int)
18
19 print(f"Sample DataFrame:\n{df.head(10)}")
20
21 # Basic groupby operations
22 print(f"\n--- Basic GroupBy Operations ---")
23
24 # Group by single column
25 dept_groups = df.groupby('Department')
26 print(f"Groups created: {dept_groups.groups}")
27
28 # Aggregation functions
29 print(f"\n--- Aggregation Functions ---")
30 print(f"Mean salary by department:\n{dept_groups['Salary'].mean()}")
31 print(f"Sum of salaries by department:\n{dept_groups['Salary'].sum()}")
32 print(f"Count of employees by department:\n{dept_groups['EmployeeID'].count()}")
33
34 # Multiple aggregations
35 print(f"\n--- Multiple Aggregations ---")
36 agg_results = dept_groups['Salary'].agg(['mean', 'std', 'min', 'max'])
37 print(f"Salary statistics by department:\n{agg_results}")
38
39 # Custom aggregation functions
40 print(f"\n--- Custom Aggregation Functions ---")
41 def salary_range(series):
42     return series.max() - series.min()
43
44 custom_agg = dept_groups['Salary'].agg({
45     'mean_salary': 'mean',
46     'salary_range': salary_range,
47     'employee_count': 'count'
48 })
49 print(f"Custom aggregations:\n{custom_agg}")
50
51 # Group by multiple columns
52 print(f"\n--- Group by Multiple Columns ---")
53 multi_group = df.groupby(['Department', 'Performance'])
54 print(f"Groups by Department and Performance:\n{multi_group.groups}")
55
56 multi_agg = multi_group['Salary'].mean()
57 print(f"Mean salary by Department and Performance:\n{multi_agg}")
58
59 # Transform operations
60 print(f"\n--- Transform Operations ---")
61 df['Salary_Mean_by_Dept'] = dept_groups['Salary'].transform('mean')
62 df['Salary_Std_by_Dept'] = dept_groups['Salary'].transform('std')
63 print(f"DataFrame with transformed columns:\n{df[['Department', 'Salary', 'Salary_Mean_by_Dept', 'Salary_Std_by_Dept']]}")
64
65 # Filter operations
66 print(f"\n--- Filter Operations ---")
67 high_performers = dept_groups.filter(lambda x: x['Performance'] == 'Excellent')
68 print(f"Departments with excellent performers:\n{high_performers}")
69
70
71
72
73

```

```

74 # Apply custom functions
75 print(f"\n--- Apply Custom Functions ---")
76 def dept_summary(group):
77     return pd.Series({
78         'avg_salary': group['Salary'].mean(),
79         'total_employees': len(group),
80         'excellent_performers': (group['Performance'] == 'Excellent')
81     })

dept_summary = dept_groups.apply(dept_summary)
print(f"Department summary:\n{dept_summary}")

```

## 2. Pivot Tables and Cross-tabulation

In [ ...

```

1 # Create a sample dataset for pivot tables
2 sales_data = {
3     'Date': pd.date_range('2024-01-01', periods=100, freq='D'),
4     'Product': np.random.choice(['Laptop', 'Phone', 'Tablet', 'Monitor']),
5     'Category': np.random.choice(['Electronics', 'Computers', 'Mobiles']),
6     'Region': np.random.choice(['North', 'South', 'East', 'West']),
7     'Sales': np.random.randint(1000, 5000, 100),
8     'Quantity': np.random.randint(1, 10, 100)
9 }
10
11
12 sales_df = pd.DataFrame(sales_data)
13 sales_df['Month'] = sales_df['Date'].dt.month
14 sales_df['Quarter'] = sales_df['Date'].dt.quarter
15
16 print(f"Sales DataFrame:\n{sales_df.head(10)}")
17
18 # Basic pivot table
19 print(f"\n--- Basic Pivot Table ---")
20 pivot_basic = sales_df.pivot_table(values='Sales', index='Product', columns='Region')
21 print(f"Sales by Product and Region:\n{pivot_basic}")
22
23 # Pivot table with multiple aggregations
24 print(f"\n--- Pivot Table with Multiple Aggregations ---")
25 pivot_multi = sales_df.pivot_table(
26     values=['Sales', 'Quantity'],
27     index='Product',
28     columns='Region',
29     aggfunc={'Sales': 'sum', 'Quantity': 'mean'}
30 )
31
32 print(f"Sales and Quantity by Product and Region:\n{pivot_multi}")
33
34 # Pivot table with margins
35 print(f"\n--- Pivot Table with Margins ---")
36 pivot_margins = sales_df.pivot_table(
37     values='Sales',
38     index='Product',
39     columns='Region',
40     aggfunc='sum',
41     margins=True,
42     margins_name='Total'
43 )
44
45 print(f"Sales with totals:\n{pivot_margins}")

```

```

46
47 # Pivot table with fill values
48 print(f"\n--- Pivot Table with Fill Values ---")
49 pivot_fill = sales_df.pivot_table(
50     values='Sales',
51     index='Product',
52     columns='Region',
53     aggfunc='sum',
54     fill_value=0
55 )
56 print(f"Sales with 0 for missing values:\n{pivot_fill}")
57
58 # Cross-tabulation
59 print(f"\n--- Cross-tabulation ---")
60 crosstab_basic = pd.crosstab(sales_df['Product'], sales_df['Region'])
61 print(f"Count of products by region:\n{crosstab_basic}")
62
63 # Cross-tabulation with normalization
64 print(f"\n--- Cross-tabulation with Normalization ---")
65 crosstab_norm = pd.crosstab(sales_df['Product'], sales_df['Region'])
66 print(f"Proportion of products by region (normalized by row):\n{crosstab_norm}")
67
68 # Cross-tabulation with margins
69 print(f"\n--- Cross-tabulation with Margins ---")
70 crosstab_margins = pd.crosstab(sales_df['Product'], sales_df['Region'], margins=True)
71 print(f"Count with totals:\n{crosstab_margins}")
72
73 # Multi-level pivot table
74 print(f"\n--- Multi-level Pivot Table ---")
75 pivot_multi_level = sales_df.pivot_table(
76     values='Sales',
77     index=['Product', 'Category'],
78     columns=['Region', 'Quarter'],
79     aggfunc='sum'
80 )
81 print(f"Sales by Product, Category, Region, and Quarter:\n{pivot_multi_level}")
82
83 # Pivot table with custom aggregation
84 print(f"\n--- Pivot Table with Custom Aggregation ---")
85 def sales_range(series):
86     return series.max() - series.min()
87
88 pivot_custom = sales_df.pivot_table(
89     values='Sales',
90     index='Product',
91     columns='Region',
92     aggfunc=['sum', 'mean', sales_range]
93 )
94 print(f"Sales with custom aggregation:\n{pivot_custom}")

```

### 3. Merging and Joining DataFrames

In [ ...

```

1 # Create sample DataFrames for merging
2 employees = pd.DataFrame({
3     'Employee_ID': [1, 2, 3, 4, 5, 6],
4     'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],

```

```

5         'Department_ID': [101, 102, 101, 103, 102, 101],
6         'Salary': [50000, 60000, 70000, 55000, 65000, 80000]
7     })
8
9     departments = pd.DataFrame({
10         'Department_ID': [101, 102, 103, 104],
11         'Department_Name': ['IT', 'HR', 'Finance', 'Marketing'],
12         'Manager': ['John', 'Jane', 'Mike', 'Sarah']
13     })
14
15     projects = pd.DataFrame({
16         'Project_ID': [1, 2, 3, 4, 5],
17         'Project_Name': ['Project A', 'Project B', 'Project C', 'Proj
18         'Employee_ID': [1, 2, 3, 4, 5],
19         'Hours': [40, 35, 50, 30, 45]
20     })
21
22     print(f"Employees DataFrame:\n{employees}")
23     print(f"\nDepartments DataFrame:\n{departments}")
24     print(f"\nProjects DataFrame:\n{projects}")
25
26     # Inner join
27     print(f"\n--- Inner Join ---")
28     inner_join = pd.merge(employees, departments, on='Department_ID',
29     print(f"Employees with Department info (inner join):\n{inner_join}")
30
31     # Left join
32     print(f"\n--- Left Join ---")
33     left_join = pd.merge(employees, departments, on='Department_ID',
34     print(f"All employees with Department info (left join):\n{left_join}")
35
36     # Right join
37     print(f"\n--- Right Join ---")
38     right_join = pd.merge(employees, departments, on='Department_ID',
39     print(f"All departments with Employee info (right join):\n{right_join}")
40
41     # Outer join
42     print(f"\n--- Outer Join ---")
43     outer_join = pd.merge(employees, departments, on='Department_ID',
44     print(f"All records from both DataFrames (outer join):\n{outer_join}")
45
46     # Multiple joins
47     print(f"\n--- Multiple Joins ---")
48     # First join employees with departments
49     emp_dept = pd.merge(employees, departments, on='Department_ID', how='left')
50     # Then join with projects
51     final_join = pd.merge(emp_dept, projects, on='Employee_ID', how='left')
52     print(f"Employees with Department and Project info:\n{final_join}")
53
54     # Join on different column names
55     print(f"\n--- Join on Different Column Names ---")
56     # Create a DataFrame with different column name
57     projects_alt = pd.DataFrame({
58         'Project_ID': [1, 2, 3, 4, 5],
59         'Project_Name': ['Project A', 'Project B', 'Project C', 'Proj
60         'Emp_ID': [1, 2, 3, 4, 5], # Different column name
61         'Hours': [40, 35, 50, 30, 45]
62     })
63
64     join_different_names = pd.merge(employees, projects_alt, left_on='Employee_ID', right_on='Emp_ID', how='left')

```

```

69     print(f"Join with different column names:\n{join_different_names}")
70
71     # Concatenation
72     print(f"\n--- Concatenation ---")
73     # Create additional employee records
74     new_employees = pd.DataFrame({
75         'Employee_ID': [7, 8, 9],
76         'Name': ['Grace', 'Henry', 'Ivy'],
77         'Department_ID': [101, 103, 102],
78         'Salary': [58000, 72000, 62000]
79     })
80
81
82     concatenated = pd.concat([employees, new_employees], ignore_index=True)
83     print(f"Concatenated employees:\n{concatenated}")
84
85     # Concatenation with different columns
86     print(f"\n--- Concatenation with Different Columns ---")
87     # Create a DataFrame with different columns
88     employee_contact = pd.DataFrame({
89         'Employee_ID': [1, 2, 3, 4, 5],
90         'Email': ['alice@company.com', 'bob@company.com', 'charlie@company.com', 'dave@company.com', 'eve@company.com'],
91         'Phone': ['555-1234', '555-5678', '555-9012', '555-3456', '555-7890']
92     })
93
94     concat_different = pd.concat([employees, employee_contact], axis=1)
95     print(f"Concatenated with different columns:\n{concat_different}")
96
97     # Append (deprecated but still useful to know)
98     print(f"\n--- Append (Alternative to Concat) ---")
99     appended = employees.append(new_employees, ignore_index=True)
100    print(f"Appended employees:\n{appended}")
101
102    # Merge with indicator
103    print(f"\n--- Merge with Indicator ---")
104    merge_indicator = pd.merge(employees, departments, on='Department_ID', indicator=True)
105    print(f"Merge with indicator:\n{merge_indicator}")
106
107    # Merge with suffixes
108    print(f"\n--- Merge with Suffixes ---")
109    # Create DataFrames with overlapping column names
110    df1 = pd.DataFrame({
111        'ID': [1, 2, 3],
112        'Value': [10, 20, 30],
113        'Name': ['A', 'B', 'C']
114    })
115
116    df2 = pd.DataFrame({
117        'ID': [1, 2, 4],
118        'Value': [15, 25, 35],
119        'Name': ['X', 'Y', 'Z']
120    })
121
122    merge_suffixes = pd.merge(df1, df2, on='ID', how='outer', suffixes=('_df1', '_df2'))
123    print(f"Merge with suffixes:\n{merge_suffixes}")

```

## 4. Data Reshaping (Melt, Pivot, Stack, Unstack)

In [ ...

```
1
2 # Create a wide DataFrame for reshaping examples
3 wide_data = {
4     'Product': ['Laptop', 'Phone', 'Tablet'],
5     'Q1_Sales': [1000, 2000, 1500],
6     'Q2_Sales': [1200, 2200, 1600],
7     'Q3_Sales': [1100, 2100, 1550],
8     'Q4_Sales': [1300, 2300, 1700],
9     'Q1_Profit': [200, 400, 300],
10    'Q2_Profit': [240, 440, 320],
11    'Q3_Profit': [220, 420, 310],
12    'Q4_Profit': [260, 460, 340]
13 }
14
15 wide_df = pd.DataFrame(wide_data)
16 print(f"Wide DataFrame:\n{wide_df}")
17
18 # Melt (wide to long)
19 print(f"\n--- Melt (Wide to Long) ---")
20 melted = pd.melt(wide_df,
21                  id_vars=['Product'],
22                  value_vars=['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales'],
23                  var_name='Quarter',
24                  value_name='Sales')
25 print(f"Melted DataFrame:\n{melted}")
26
27 # Melt with multiple value columns
28 print(f"\n--- Melt with Multiple Value Columns ---")
29 melted_multi = pd.melt(wide_df,
30                        id_vars=['Product'],
31                        value_vars=['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales',
32                                   'Q1_Profit', 'Q2_Profit', 'Q3_Profit', 'Q4_Profit'],
33                        var_name='Metric_Quarter',
34                        value_name='Value')
35 print(f"Melted with multiple columns:\n{melted_multi}")
36
37 # Pivot (long to wide)
38 print(f"\n--- Pivot (Long to Wide) ---")
39 # First create a long DataFrame
40 long_data = {
41     'Product': ['Laptop', 'Laptop', 'Laptop', 'Laptop', 'Phone', 'Tablet'],
42     'Quarter': ['Q1', 'Q2', 'Q3', 'Q4', 'Q1', 'Q2', 'Q3', 'Q4'],
43     'Sales': [1000, 1200, 1100, 1300, 2000, 2200, 2100, 2300],
44     'Profit': [200, 240, 220, 260, 400, 440, 420, 460]
45 }
46
47 long_df = pd.DataFrame(long_data)
48 print(f"Long DataFrame:\n{long_df}")
49
50 # Pivot to wide format
51 pivoted = long_df.pivot(index='Product', columns='Quarter', value='Sales')
52 print(f"Pivoted DataFrame:\n{pivoted}")
53
54 # Pivot with multiple values
55 pivot_multi = long_df.pivot(index='Product', columns='Quarter', value='Sales')
56 print(f"Pivoted with multiple values:\n{pivot_multi}")
57
58 # Stack and Unstack
```

```

63 print(f"\n--- Stack and Unstack ---")
64 # Create a DataFrame with MultiIndex
65 multi_index_data = {
66     'Sales': [[1000, 1200, 1100, 1300], [2000, 2200, 2100, 2300],
67     'Profit': [[200, 240, 220, 260], [400, 440, 420, 460], [300,
68     ]
69
70 multi_df = pd.DataFrame(multi_index_data, index=['Laptop', 'Phone
71 multi_df.columns = pd.MultiIndex.from_tuples([('Q1', 'Sales'), ('
72                                     ('Q1', 'Profit'), ('Q
73
74 print(f"MultiIndex DataFrame:\n{multi_df}")
75
76 # Stack (columns to index)
77 stacked = multi_df.stack()
78 print(f"Stacked DataFrame:\n{stacked}")
79
80 # Unstack (index to columns)
81 unstacked = stacked.unstack()
82 print(f"Unstacked DataFrame:\n{unstacked}")
83
84 # Transpose
85 print(f"\n--- Transpose ---")
86 transposed = wide_df.set_index('Product').T
87 print(f"Transposed DataFrame:\n{transposed}")
88
89 # Reshape using pivot_table
90 print(f"\n--- Pivot Table for Reshaping ---")
91 # Create a more complex long DataFrame
92 complex_long = pd.DataFrame({
93     'Product': ['Laptop', 'Laptop', 'Laptop', 'Laptop', 'Phone',
94     'Quarter': ['Q1', 'Q2', 'Q3', 'Q4', 'Q1', 'Q2', 'Q3', 'Q4'] *
95     'Metric': ['Sales'] * 8 + ['Profit'] * 8,
96     'Value': [1000, 1200, 1100, 1300, 2000, 2200, 2100, 2300,
97               200, 240, 220, 260, 400, 440, 420, 460]
98 })
99
100
101 print(f"Complex long DataFrame:\n{complex_long}")
102
103 # Pivot table for reshaping
104 pivot_table_reshape = complex_long.pivot_table(
105     index='Product',
106     columns=['Quarter', 'Metric'],
107     values='Value'
108 )
109
110 print(f"Pivot table reshape:\n{pivot_table_reshape}")
111
112 # Melt with multiple id variables
113 print(f"\n--- Melt with Multiple ID Variables ---")
114 # Add a category column
115 wide_df_with_category = wide_df.copy()
116 wide_df_with_category['Category'] = ['Electronics', 'Mobile', 'El
117
118 melted_category = pd.melt(wide_df_with_category,
119                           id_vars=['Product', 'Category'],
120                           value_vars=['Q1_Sales', 'Q2_Sales', 'Q3_
121                           var_name='Quarter',
122                           value_name='Sales')
123
124 print(f"Melted with category:\n{melted_category}")

```



## 5. Concatenating DataFrames

In [ ...

```
1  # Create sample DataFrames for concatenation examples
2
3  df1 = pd.DataFrame({
4      'A': [1, 2, 3],
5      'B': [4, 5, 6],
6      'C': [7, 8, 9]
7  })
8
9  df2 = pd.DataFrame({
10     'A': [10, 11, 12],
11     'B': [13, 14, 15],
12     'C': [16, 17, 18]
13 })
14
15 df3 = pd.DataFrame({
16     'A': [19, 20, 21],
17     'B': [22, 23, 24],
18     'C': [25, 26, 27]
19 })
20
21
22 print(f"DataFrame 1:\n{df1}")
23 print(f"\nDataFrame 2:\n{df2}")
24 print(f"\nDataFrame 3:\n{df3}")
25
26 # Basic concatenation (vertical)
27 print(f"\n--- Basic Concatenation (Vertical) ---")
28 concatenated_vertical = pd.concat([df1, df2, df3], ignore_index=True)
29 print(f"Concatenated vertically:\n{concatenated_vertical}")
30
31 # Concatenation with keys
32 print(f"\n--- Concatenation with Keys ---")
33 concatenated_keys = pd.concat([df1, df2, df3], keys=['Group1', 'Group2', 'Group3'])
34 print(f"Concatenated with keys:\n{concatenated_keys}")
35
36 # Horizontal concatenation
37 print(f"\n--- Horizontal Concatenation ---")
38 df4 = pd.DataFrame({
39     'D': [100, 101, 102],
40     'E': [103, 104, 105]
41 })
42
43
44 concatenated_horizontal = pd.concat([df1, df4], axis=1)
45 print(f"Concatenated horizontally:\n{concatenated_horizontal}")
46
47 # Concatenation with different columns
48 print(f"\n--- Concatenation with Different Columns ---")
49 df5 = pd.DataFrame({
50     'A': [30, 31, 32],
51     'D': [33, 34, 35], # Different column
52     'E': [36, 37, 38]  # Different column
53 })
54
55
56 concatenated_different = pd.concat([df1, df5], ignore_index=True)
57 print(f"Concatenated with different columns:\n{concatenated_different}")
58
```

```

59 # Concatenation with sort
60 print(f"\n--- Concatenation with Sort ---")
61 concatenated_sorted = pd.concat([df1, df5], ignore_index=True, so
62 print(f"Concatenated with sorted columns:\n{concatenated_sorted}")
63
64 # Concatenation with join parameter
65 print(f"\n--- Concatenation with Join Parameter ---")
66 # Inner join (only common columns)
67 concatenated_inner = pd.concat([df1, df5], join='inner', ignore_i
68 print(f"Concatenated with inner join:\n{concatenated_inner}")
69
70 # Outer join (all columns)
71 concatenated_outer = pd.concat([df1, df5], join='outer', ignore_i
72 print(f"Concatenated with outer join:\n{concatenated_outer}")
73
74 # Concatenation with Series
75 print(f"\n--- Concatenation with Series ---")
76 series1 = pd.Series([100, 200, 300], name='Series1')
77 series2 = pd.Series([400, 500, 600], name='Series2')
78
79 concatenated_series = pd.concat([series1, series2], ignore_index=
80 print(f"Concatenated series:\n{concatenated_series}")
81
82 # Concatenation to create DataFrame from Series
83 print(f"\n--- DataFrame from Series ---")
84 df_from_series = pd.concat([series1, series2], axis=1)
85 print(f"DataFrame from series:\n{df_from_series}")
86
87 # Concatenation with MultiIndex
88 print(f"\n--- Concatenation with MultiIndex ---")
89 df_multi1 = pd.DataFrame({
90     'Value': [1, 2, 3, 4]
91 }, index=pd.MultiIndex.from_tuples([('A', 'X'), ('A', 'Y'), ('B',
92
93 df_multi2 = pd.DataFrame({
94     'Value': [5, 6, 7, 8]
95 }, index=pd.MultiIndex.from_tuples([('C', 'X'), ('C', 'Y'), ('D',
96
97 concatenated_multi = pd.concat([df_multi1, df_multi2])
98 print(f"Concatenated with MultiIndex:\n{concatenated_multi}")
99
100 # Concatenation with different index levels
101 print(f"\n--- Concatenation with Different Index Levels ---")
102 df_level1 = pd.DataFrame({
103     'Value': [1, 2, 3]
104 }, index=['A', 'B', 'C'])
105
106 df_level2 = pd.DataFrame({
107     'Value': [4, 5, 6]
108 }, index=pd.MultiIndex.from_tuples([('X', 1), ('X', 2), ('Y', 1)]
109
110 concatenated_levels = pd.concat([df_level1, df_level2])
111 print(f"Concatenated with different index levels:\n{concatenated_
112
113 # Performance comparison
114 print(f"\n--- Performance Comparison ---")
115 import time
116
117 # Large DataFrames for performance testing
118 large_df1 = pd.DataFrame(np.random.randn(1000, 3), columns=['A',

```

```

122 large_df2 = pd.DataFrame(np.random.randn(1000, 3), columns=['A',
123
# Time concatenation
start_time = time.time()
result = pd.concat([large_df1, large_df2], ignore_index=True)
end_time = time.time()
print(f"Concatenation time: {end_time - start_time:.4f} seconds")
print(f"Result shape: {result.shape}")

```

## 6. Time Series Operations

In [ ...

```

1 # Create time series data
2 dates = pd.date_range('2024-01-01', periods=100, freq='D')
3 ts_data = {
4     'Date': dates,
5     'Stock_Price': 100 + np.cumsum(np.random.randn(100) * 0.5),
6     'Volume': np.random.randint(1000, 10000, 100),
7     'High': 100 + np.cumsum(np.random.randn(100) * 0.5) + np.random.randn(100) * 0.5,
8     'Low': 100 + np.cumsum(np.random.randn(100) * 0.5) - np.random.randn(100) * 0.5
9 }
10
11
12 ts_df = pd.DataFrame(ts_data)
13 ts_df.set_index('Date', inplace=True)
14
15 print(f"Time Series DataFrame:\n{ts_df.head(10)}")
16 print(f"\nDataFrame info:\n{ts_df.info()}")
17
18 # Basic time series operations
19 print(f"\n--- Basic Time Series Operations ---")
20
21 # Date range operations
22 print(f"Date range: {ts_df.index.min()} to {ts_df.index.max()}")
23 print(f"Number of days: {len(ts_df)}")
24
25 # Time-based indexing
26 print(f"\n--- Time-based Indexing ---")
27 print(f"January 2024 data:\n{ts_df['2024-01'].head()}")
28 print(f"First week data:\n{ts_df['2024-01-01':'2024-01-07']}")
29
30 # Resampling
31 print(f"\n--- Resampling ---")
32 # Daily to weekly
33 weekly_data = ts_df.resample('W').agg({
34     'Stock_Price': 'last',
35     'Volume': 'sum',
36     'High': 'max',
37     'Low': 'min'
38 })
39
40 print(f"Weekly data:\n{weekly_data.head()}")
41
42 # Daily to monthly
43 monthly_data = ts_df.resample('M').agg({
44     'Stock_Price': 'last',
45     'Volume': 'sum',
46     'High': 'max',
47     'Low': 'min'
48 })

```

```

49     })
50     print(f"Monthly data:\n{monthly_data.head()}")
51
52     # Time shifting
53     print(f"\n--- Time Shifting ---")
54     ts_df['Price_1Day_Lag'] = ts_df['Stock_Price'].shift(1)
55     ts_df['Price_1Day_Lead'] = ts_df['Stock_Price'].shift(-1)
56     ts_df['Price_1Week_Lag'] = ts_df['Stock_Price'].shift(7)
57
58     print(f"Time shifted data:\n{ts_df[['Stock_Price', 'Price_1Day_La
59
60     # Time differences
61     print(f"\n--- Time Differences ---")
62     ts_df['Price_Change'] = ts_df['Stock_Price'].diff()
63     ts_df['Price_Pct_Change'] = ts_df['Stock_Price'].pct_change()
64     ts_df['Price_Change_1Week'] = ts_df['Stock_Price'].diff(7)
65
66     print(f"Price changes:\n{ts_df[['Stock_Price', 'Price_Change', 'P
67
68     # Rolling operations
69     print(f"\n--- Rolling Operations ---")
70     ts_df['Price_MA_5'] = ts_df['Stock_Price'].rolling(window=5).mean
71     ts_df['Price_MA_20'] = ts_df['Stock_Price'].rolling(window=20).me
72     ts_df['Price_Std_10'] = ts_df['Stock_Price'].rolling(window=10).s
73     ts_df['Price_Max_7'] = ts_df['Stock_Price'].rolling(window=7).max
74     ts_df['Price_Min_7'] = ts_df['Stock_Price'].rolling(window=7).min
75
76     print(f"Rolling statistics:\n{ts_df[['Stock_Price', 'Price_MA_5',
77
78     # Expanding operations
79     print(f"\n--- Expanding Operations ---")
80     ts_df['Price_Expanding_Mean'] = ts_df['Stock_Price'].expanding().
81     ts_df['Price_Expanding_Std'] = ts_df['Stock_Price'].expanding().s
82     ts_df['Price_Expanding_Max'] = ts_df['Stock_Price'].expanding().m
83
84     print(f"Expanding statistics:\n{ts_df[['Stock_Price', 'Price_Exp
85
86     # Time series decomposition
87     print(f"\n--- Time Series Decomposition ---")
88     from statsmodels.tsa.seasonal import seasonal_decompose
89
90     # Create a more realistic time series with trend and seasonality
91     np.random.seed(42)
92     t = np.arange(100)
93     trend = 0.1 * t
94     seasonal = 2 * np.sin(2 * np.pi * t / 7) # Weekly seasonality
95     noise = np.random.randn(100) * 0.5
96     ts_values = 100 + trend + seasonal + noise
97
98     ts_decomp = pd.DataFrame({
99         'Date': dates,
100         'Value': ts_values
101     }).set_index('Date')
102
103     # Decompose the time series
104     decomposition = seasonal_decompose(ts_decomp['Value'], model='add
105     print(f"Decomposition components:\n{decomposition.trend.head()}")
106     print(f"Seasonal component:\n{decomposition.seasonal.head()}")
107     print(f"Residual component:\n{decomposition.resid.head()}")
108
109
110
111
112

```

```

113 # Time series frequency conversion
114 print(f"\n--- Frequency Conversion ---")
115 # Convert daily to business days
116 business_days = ts_df.asfreq('B') # Business days
117 print(f"Business days data:\n{business_days.head()}")
118
119 # Convert to hourly (with forward fill)
120 hourly_data = ts_df.asfreq('H', method='ffill')
121 print(f"Hourly data (first 24 hours):\n{hourly_data.head(24)}")
122
123 # Time zone operations
124 print(f"\n--- Time Zone Operations ---")
125 # Create timezone-aware data
126 ts_df_tz = ts_df.copy()
127 ts_df_tz.index = ts_df_tz.index.tz_localize('UTC')
128 print(f"UTC timezone data:\n{ts_df_tz.head()}")
129
130 # Convert to different timezone
131 ts_df_est = ts_df_tz.tz_convert('US/Eastern')
132 print(f"EST timezone data:\n{ts_df_est.head()}")
133
134 # Time series interpolation
135 print(f"\n--- Time Series Interpolation ---")
136 # Create data with missing values
137 ts_missing = ts_df.copy()
138 ts_missing.loc[ts_missing.index[10:15], 'Stock_Price'] = np.nan
139 ts_missing.loc[ts_missing.index[30:35], 'Stock_Price'] = np.nan
140
141 print(f"Data with missing values:\n{ts_missing[['Stock_Price']].head(4)}")
142
143 # Interpolate missing values
144 ts_interpolated = ts_missing.interpolate(method='linear')
145 print(f"Interpolated data:\n{ts_interpolated[['Stock_Price']].head(4)}")
146
147 # Different interpolation methods
148 ts_spline = ts_missing.interpolate(method='spline', order=2)
149 ts_polynomial = ts_missing.interpolate(method='polynomial', order=3)
150
151 print(f"Spline interpolation:\n{ts_spline[['Stock_Price']].head(4)}")
152
153 # Time series aggregation
154 print(f"\n--- Time Series Aggregation ---")
155 # Group by time periods
156 ts_df['Year'] = ts_df.index.year
157 ts_df['Month'] = ts_df.index.month
158 ts_df['DayOfWeek'] = ts_df.index.dayofweek
159 ts_df['Quarter'] = ts_df.index.quarter
160
161 # Aggregate by year
162 yearly_agg = ts_df.groupby('Year').agg({
163     'Stock_Price': ['mean', 'std', 'min', 'max'],
164     'Volume': 'sum'
165 })
166 print(f"Yearly aggregation:\n{yearly_agg}")
167
168 # Aggregate by quarter
169 quarterly_agg = ts_df.groupby('Quarter').agg({
170     'Stock_Price': 'mean',
171     'Volume': 'sum'
172 })

```

```

177     print(f"Quarterly aggregation:\n{quarterly_agg}")
178
179     # Aggregate by day of week
180     dow_agg = ts_df.groupby('DayOfWeek').agg({
181         'Stock_Price': 'mean',
182         'Volume': 'mean'
183     })
184     print(f"Day of week aggregation:\n{dow_agg}")
185
186     # Time series plotting
187     print(f"\n--- Time Series Plotting ---")
188     import matplotlib.pyplot as plt
189
190     # Create a simple plot
191     plt.figure(figsize=(12, 6))
192     plt.subplot(2, 1, 1)
193     plt.plot(ts_df.index, ts_df['Stock_Price'], label='Stock Price')
194     plt.plot(ts_df.index, ts_df['Price_MA_5'], label='5-day MA')
195     plt.plot(ts_df.index, ts_df['Price_MA_20'], label='20-day MA')
196     plt.title('Stock Price with Moving Averages')
197     plt.legend()
198
199     plt.subplot(2, 1, 2)
200     plt.plot(ts_df.index, ts_df['Volume'])
201     plt.title('Trading Volume')
202     plt.tight_layout()
203     plt.show()
204
205     print("Time series plots created successfully!")

```

## 7. Window Functions and Rolling Operations

In [ ...

```

1     # Create sample data for window functions
2     np.random.seed(42)
3     window_data = {
4         'Date': pd.date_range('2024-01-01', periods=50, freq='D'),
5         'Stock_Price': 100 + np.cumsum(np.random.randn(50) * 0.5),
6         'Volume': np.random.randint(1000, 10000, 50),
7         'Company': ['AAPL'] * 25 + ['GOOGL'] * 25
8     }
9
10
11     window_df = pd.DataFrame(window_data)
12     window_df.set_index('Date', inplace=True)
13
14     print(f"Window Functions DataFrame:\n{window_df.head(10)}")
15
16     # Basic rolling operations
17     print(f"\n--- Basic Rolling Operations ---")
18
19     # Rolling mean
20     window_df['Price_MA_5'] = window_df['Stock_Price'].rolling(window=5).mean()
21     window_df['Price_MA_10'] = window_df['Stock_Price'].rolling(window=10).mean()
22     window_df['Price_MA_20'] = window_df['Stock_Price'].rolling(window=20).mean()
23
24     print(f"Rolling means:\n{window_df[['Stock_Price', 'Price_MA_5', 'Price_MA_10', 'Price_MA_20']].head(10)}")
25
26

```

```

27 # Rolling standard deviation
28 window_df['Price_Std_5'] = window_df['Stock_Price'].rolling(windo
29 window_df['Price_Std_10'] = window_df['Stock_Price'].rolling(wind
30
31 print(f"Rolling standard deviations:\n{window_df[['Stock_Price',
32
33 # Rolling min/max
34 window_df['Price_Min_5'] = window_df['Stock_Price'].rolling(windo
35 window_df['Price_Max_5'] = window_df['Stock_Price'].rolling(windo
36 window_df['Price_Range_5'] = window_df['Price_Max_5'] - window_df
37
38 print(f"Rolling min/max:\n{window_df[['Stock_Price', 'Price_Min_5
39
40 # Rolling sum
41 window_df['Volume_Sum_5'] = window_df['Volume'].rolling(window=5)
42 window_df['Volume_Sum_10'] = window_df['Volume'].rolling(window=1
43
44 print(f"Rolling volume sums:\n{window_df[['Volume', 'Volume_Sum_5
45
46 # Rolling quantiles
47 window_df['Price_Q25_10'] = window_df['Stock_Price'].rolling(wind
48 window_df['Price_Q75_10'] = window_df['Stock_Price'].rolling(wind
49 window_df['Price_IQR_10'] = window_df['Price_Q75_10'] - window_df
50
51 print(f"Rolling quantiles:\n{window_df[['Stock_Price', 'Price_Q25
52
53 # Rolling apply with custom functions
54 print(f"\n--- Rolling Apply with Custom Functions ---")
55
56 def custom_volatility(series):
57     return series.std() / series.mean()
58
59 def custom_skewness(series):
60     return series.skew()
61
62 def custom_kurtosis(series):
63     return series.kurtosis()
64
65 window_df['Price_Volatility_10'] = window_df['Stock_Price'].rolli
66 window_df['Price_Skewness_10'] = window_df['Stock_Price'].rolling
67 window_df['Price_Kurtosis_10'] = window_df['Stock_Price'].rolling
68
69 print(f"Custom rolling functions:\n{window_df[['Stock_Price', 'Pr
70
71 # Rolling window with different alignments
72 print(f"\n--- Rolling Window Alignments ---")
73
74 # Center alignment (default)
75 window_df['Price_MA_Center'] = window_df['Stock_Price'].rolling(w
76
77 # Right alignment
78 window_df['Price_MA_Right'] = window_df['Stock_Price'].rolling(wi
79
80 # Left alignment
81 window_df['Price_MA_Left'] = window_df['Stock_Price'].rolling(win
82
83 print(f"Different alignments:\n{window_df[['Stock_Price', 'Price_
84
85 # Expanding operations
86 print(f"\n--- Expanding Operations ---")
87
88
89
90

```



```

91
92     # Expanding mean
93     window_df['Price_Exp_Mean'] = window_df['Stock_Price'].expanding(
94     window_df['Price_Exp_Std'] = window_df['Stock_Price'].expanding()
95     window_df['Price_Exp_Min'] = window_df['Stock_Price'].expanding()
96     window_df['Price_Exp_Max'] = window_df['Stock_Price'].expanding()
97
98     print(f"Expanding operations:\n{window_df[['Stock_Price', 'Price_
99
100     # Expanding with custom functions
101     window_df['Price_Exp_Volatility'] = window_df['Stock_Price'].expa
102     window_df['Price_Exp_Skewness'] = window_df['Stock_Price'].expand
103
104     print(f"Expanding custom functions:\n{window_df[['Stock_Price', '
105
106     # Exponentially weighted operations
107     print(f"\n--- Exponentially Weighted Operations ---")
108
109     # Exponentially weighted mean
110     window_df['Price_EWM_5'] = window_df['Stock_Price'].ewm(span=5).m
111     window_df['Price_EWM_10'] = window_df['Stock_Price'].ewm(span=10)
112     window_df['Price_EWM_20'] = window_df['Stock_Price'].ewm(span=20)
113
114     print(f"Exponentially weighted means:\n{window_df[['Stock_Price',
115
116     # Exponentially weighted standard deviation
117     window_df['Price_EWM_Std_10'] = window_df['Stock_Price'].ewm(span
118     window_df['Price_EWM_Var_10'] = window_df['Stock_Price'].ewm(span
119
120     print(f"Exponentially weighted std/var:\n{window_df[['Stock_Price
121
122     # Exponentially weighted with different halflife
123     window_df['Price_EWM_HL_5'] = window_df['Stock_Price'].ewm(halfli
124     window_df['Price_EWM_HL_10'] = window_df['Stock_Price'].ewm(halfli
125
126     print(f"Exponentially weighted with halflife:\n{window_df[['Stock
127
128     # Rolling window with groupby
129     print(f"\n--- Rolling Window with GroupBy ---")
130
131     # Group by company and apply rolling operations
132     window_df['Price_MA_5_Grouped'] = window_df.groupby('Company')['S
133     window_df['Price_Std_5_Grouped'] = window_df.groupby('Company')['
134
135     print(f"Grouped rolling operations:\n{window_df[['Company', 'Stoc
136
137     # Rolling window with time-based windows
138     print(f"\n--- Time-based Rolling Windows ---")
139
140     # Rolling window based on time periods
141     window_df['Price_MA_7D'] = window_df['Stock_Price'].rolling(windo
142     window_df['Price_MA_14D'] = window_df['Stock_Price'].rolling(wind
143     window_df['Price_MA_30D'] = window_df['Stock_Price'].rolling(wind
144
145     print(f"Time-based rolling means:\n{window_df[['Stock_Price', 'Pr
146
147     # Rolling window with business days
148     window_df['Price_MA_5B'] = window_df['Stock_Price'].rolling(windo
149     window_df['Price_MA_10B'] = window_df['Stock_Price'].rolling(wind
150
151
152
153
154

```



```

155     print(f"Business day rolling means:\n{window_df[['Stock_Price', '
156
157     # Rolling window with custom offsets
158     print(f"\n--- Custom Offset Rolling Windows ---")
159
160     # Rolling window with custom offset
161     window_df['Price_MA_Custom'] = window_df['Stock_Price'].rolling(w
162     window_df['Price_MA_Custom_2W'] = window_df['Stock_Price'].rollin
163
164     print(f"Custom offset rolling means:\n{window_df[['Stock_Price',
165
166     # Rolling window with min_periods
167     print(f"\n--- Rolling Window with min_periods ---")
168
169     # Rolling mean with minimum periods
170     window_df['Price_MA_Min3'] = window_df['Stock_Price'].rolling(win
171     window_df['Price_MA_Min1'] = window_df['Stock_Price'].rolling(win
172
173     print(f"Rolling means with min_periods:\n{window_df[['Stock_Price
174
175     # Rolling window with different aggregation functions
176     print(f"\n--- Rolling Window Aggregations ---")
177
178     # Multiple aggregations in one call
179     rolling_agg = window_df['Stock_Price'].rolling(window=10).agg(['m
180     print(f"Rolling aggregations:\n{rolling_agg.head(15)}")
181
182     # Rolling window with custom aggregation
183     def custom_agg(series):
184         return {
185             'mean': series.mean(),
186             'std': series.std(),
187             'range': series.max() - series.min(),
188             'volatility': series.std() / series.mean()
189         }
190
191     rolling_custom = window_df['Stock_Price'].rolling(window=10).appl
192     print(f"Custom rolling aggregation:\n{rolling_custom.head(15)}")
193
194     # Rolling window with different data types
195     print(f"\n--- Rolling Window with Different Data Types ---")
196
197     # Rolling count of non-null values
198     window_df['Volume_Count_5'] = window_df['Volume'].rolling(window=
199     window_df['Volume_Count_10'] = window_df['Volume'].rolling(window
200
201     # Rolling sum
202     window_df['Volume_Sum_5'] = window_df['Volume'].rolling(window=5)
203     window_df['Volume_Sum_10'] = window_df['Volume'].rolling(window=1
204
205     # Rolling mean
206     window_df['Volume_Mean_5'] = window_df['Volume'].rolling(window=5
207     window_df['Volume_Mean_10'] = window_df['Volume'].rolling(window=
208
209     print(f"Volume rolling operations:\n{window_df[['Volume', 'Volume
210
211     # Rolling window with missing data handling
212     print(f"\n--- Rolling Window with Missing Data ---")
213
214     # Create data with missing values

```

```

219 window_missing = window_df.copy()
220 window_missing.loc[window_missing.index[10:15], 'Stock_Price'] =
221 window_missing.loc[window_missing.index[25:30], 'Stock_Price'] =
222
223 # Rolling mean with different missing data handling
224 window_missing['Price_MA_5_Default'] = window_missing['Stock_Pric
225 window_missing['Price_MA_5_Min3'] = window_missing['Stock_Price']
226 window_missing['Price_MA_5_Min1'] = window_missing['Stock_Price']
227
228 print(f"Rolling with missing data:\n{window_missing[['Stock_Price
229
230 # Rolling window performance comparison
231 print(f"\n--- Rolling Window Performance ---")
232 import time
233
234 # Large dataset for performance testing
235 large_data = pd.DataFrame({
236     'Date': pd.date_range('2020-01-01', periods=1000, freq='D'),
237     'Price': 100 + np.cumsum(np.random.randn(1000) * 0.5)
238 }).set_index('Date')
239
240 # Time different rolling operations
241 start_time = time.time()
242 large_data['MA_5'] = large_data['Price'].rolling(window=5).mean()
243 end_time = time.time()
244 print(f"Rolling mean (5): {end_time - start_time:.4f} seconds")
245
246 start_time = time.time()
247 large_data['MA_20'] = large_data['Price'].rolling(window=20).mean()
248 end_time = time.time()
249 print(f"Rolling mean (20): {end_time - start_time:.4f} seconds")
250
251 start_time = time.time()
252 large_data['EWM_10'] = large_data['Price'].ewm(span=10).mean()
253 end_time = time.time()
254 print(f"Exponentially weighted mean: {end_time - start_time:.4f}
255
256 print(f"Large dataset shape: {large_data.shape}")
257 print(f"Sample results:\n{large_data[['Price', 'MA_5', 'MA_20', '
258
259 print("\nWindow functions and rolling operations completed succes

```

## 8. Advanced Filtering and Querying

In [ ...

```

1 # Create comprehensive dataset for advanced filtering
2 np.random.seed(42)
3 filter_data = {
4     'Employee_ID': range(1, 101),
5     'Name': [f'Employee_{i}' for i in range(1, 101)],
6     'Department': np.random.choice(['IT', 'HR', 'Finance', 'Marke
7     'Salary': np.random.normal(60000, 15000, 100).astype(int),
8     'Experience': np.random.randint(1, 25, 100),
9     'Performance': np.random.choice(['Excellent', 'Good', 'Averag
10    'Location': np.random.choice(['New York', 'Los Angeles', 'Chi
11    'Age': np.random.randint(22, 65, 100),
12    'Bonus': np.random.normal(5000, 2000, 100).astype(int),
13

```

```

14         'Start_Date': pd.date_range('2020-01-01', periods=100, freq='
15     }
16
17     filter_df = pd.DataFrame(filter_data)
18     print(f"Filtering DataFrame:\n{filter_df.head(10)}")
19     print(f"\nDataFrame shape: {filter_df.shape}")
20
21     # Basic filtering with boolean indexing
22     print(f"\n--- Basic Boolean Indexing ---")
23
24     # Simple conditions
25     high_salary = filter_df[filter_df['Salary'] > 70000]
26     print(f"High salary employees (>70k): {len(high_salary)}")
27     print(f"Sample high salary employees:\n{high_salary[['Name', 'Dep
28
29
30     # Multiple conditions with AND (&)
31     it_high_salary = filter_df[(filter_df['Department'] == 'IT') & (f
32     print(f"\nIT employees with high salary: {len(it_high_salary)}")
33     print(f"Sample IT high salary employees:\n{it_high_salary[['Name'
34
35
36     # Multiple conditions with OR (|)
37     it_or_finance = filter_df[(filter_df['Department'] == 'IT') | (fi
38     print(f"\nIT or Finance employees: {len(it_or_finance)}")
39     print(f"Sample IT or Finance employees:\n{it_or_finance[['Name',
40
41
42     # Complex conditions with parentheses
43     complex_filter = filter_df[
44         ((filter_df['Department'] == 'IT') | (filter_df['Department']
45         (filter_df['Salary'] > 60000) &
46         (filter_df['Experience'] > 5)
47     ]
48     print(f"\nComplex filter (IT/Finance, >60k salary, >5 years exp):
49     print(f"Sample complex filter results:\n{complex_filter[['Name',
50
51
52     # Using isin() for multiple values
53     print(f"\n--- Using isin() for Multiple Values ---")
54
55     # Filter by multiple departments
56     selected_depts = filter_df[filter_df['Department'].isin(['IT', 'H
57     print(f"Employees in IT, HR, or Finance: {len(selected_depts)}")
58
59     # Filter by multiple performance levels
60     good_performers = filter_df[filter_df['Performance'].isin(['Excel
61     print(f"Good performers: {len(good_performers)}")
62
63     # Filter by multiple locations
64     major_cities = filter_df[filter_df['Location'].isin(['New York',
65     print(f"Employees in major cities: {len(major_cities)}")
66
67     # Using query() method
68     print(f"\n--- Using query() Method ---")
69
70     # Simple query
71     query_result1 = filter_df.query('Salary > 70000')
72     print(f"Query: Salary > 70000, Results: {len(query_result1)}")
73
74     # Complex query with multiple conditions
75     query_result2 = filter_df.query('Department == "IT" and Salary >
76     print(f"Query: IT department, >60k salary, >5 years exp, Results:
77

```

```

78 # Query with OR conditions
79 query_result3 = filter_df.query('Department in ["IT", "Finance"]
80 print(f"Query: IT or Finance, >50k salary, Results: {len(query_re
81
82 # Query with string methods
83 query_result4 = filter_df.query('Name.str.contains("Employee_1")'
84 print(f"Query: Name contains 'Employee_1', Results: {len(query_re
85
86 # Query with date filtering
87 query_result5 = filter_df.query('Start_Date >= "2020-06-01"')
88 print(f"Query: Started after June 2020, Results: {len(query_resul
89
90
91 # Advanced string filtering
92 print(f"\n--- Advanced String Filtering ---")
93
94 # Using str.contains()
95 names_with_1 = filter_df[filter_df['Name'].str.contains('Employee
96 print(f"Names containing 'Employee_1': {len(names_with_1)}")
97
98 # Using str.startswith()
99 names_starting_with = filter_df[filter_df['Name'].str.startswith(
100 print(f"Names starting with 'Employee_1': {len(names_starting_wit
101
102 # Using str.endswith()
103 names_ending_with = filter_df[filter_df['Name'].str.endswith('0')
104 print(f"Names ending with '0': {len(names_ending_with)}")
105
106 # Using str.match() with regex
107 names_matching = filter_df[filter_df['Name'].str.match(r'Employee
108 print(f"Names matching pattern 'Employee_[1-5]\\d': {len(names_ma
109
110 # Case-insensitive string filtering
111 names_case_insensitive = filter_df[filter_df['Name'].str.contains
112 print(f"Names containing 'employee' (case-insensitive): {len(name
113
114 # Numeric filtering with ranges
115 print(f"\n--- Numeric Filtering with Ranges ---")
116
117 # Using between()
118 salary_range = filter_df[filter_df['Salary'].between(50000, 70000
119 print(f"Salary between 50k-70k: {len(salary_range)}")
120
121 # Using multiple between conditions
122 age_salary_range = filter_df[
123     filter_df['Age'].between(30, 50) &
124     filter_df['Salary'].between(60000, 80000)
125 ]
126 print(f"Age 30-50 and salary 60k-80k: {len(age_salary_range)}")
127
128 # Using quantile-based filtering
129 salary_75th = filter_df['Salary'].quantile(0.75)
130 high_earners = filter_df[filter_df['Salary'] > salary_75th]
131 print(f"Top 25% earners (>{salary_75th:.0f}): {len(high_earners)}")
132
133 # Using percentile-based filtering
134 salary_90th = filter_df['Salary'].quantile(0.9)
135 top_earners = filter_df[filter_df['Salary'] > salary_90th]
136 print(f"Top 10% earners (>{salary_90th:.0f}): {len(top_earners)}")
137
138 # Date filtering
139
140
141

```

```

142 print(f"\n--- Date Filtering ---")
143
144 # Filter by year
145 employees_2020 = filter_df[filter_df['Start_Date'].dt.year == 2020]
146 print(f"Employees who started in 2020: {len(employees_2020)}")
147
148 # Filter by month
149 employees_jan = filter_df[filter_df['Start_Date'].dt.month == 1]
150 print(f"Employees who started in January: {len(employees_jan)}")
151
152 # Filter by date range
153 date_range = filter_df[
154     (filter_df['Start_Date'] >= '2020-06-01') &
155     (filter_df['Start_Date'] <= '2020-12-31')
156 ]
157 print(f"Employees who started in H2 2020: {len(date_range)}")
158
159 # Filter by day of week
160 employees_monday = filter_df[filter_df['Start_Date'].dt.dayofweek == 0]
161 print(f"Employees who started on Monday: {len(employees_monday)}")
162
163 # Filter by quarter
164 employees_q1 = filter_df[filter_df['Start_Date'].dt.quarter == 1]
165 print(f"Employees who started in Q1: {len(employees_q1)}")
166
167 # Null value filtering
168 print(f"\n--- Null Value Filtering ---")
169
170 # Create data with null values
171 filter_df_with_nulls = filter_df.copy()
172 filter_df_with_nulls.loc[filter_df_with_nulls.index[10:15], 'Bonus'] = None
173 filter_df_with_nulls.loc[filter_df_with_nulls.index[20:25], 'Experience'] = None
174
175 # Filter non-null values
176 non_null_bonus = filter_df_with_nulls[filter_df_with_nulls['Bonus'].notnull()]
177 print(f"Employees with non-null bonus: {len(non_null_bonus)}")
178
179 # Filter null values
180 null_bonus = filter_df_with_nulls[filter_df_with_nulls['Bonus'].isnull()]
181 print(f"Employees with null bonus: {len(null_bonus)}")
182
183 # Filter multiple columns for nulls
184 any_null = filter_df_with_nulls[filter_df_with_nulls[['Bonus', 'Experience']].isnull().any(axis=1)]
185 print(f"Employees with any null in Bonus or Experience: {len(any_null)}")
186
187 # Filter all columns for nulls
188 all_null = filter_df_with_nulls[filter_df_with_nulls[['Bonus', 'Experience']].isnull().all(axis=1)]
189 print(f"Employees with all null in Bonus and Experience: {len(all_null)}")
190
191 # Advanced filtering with custom functions
192 print(f"\n--- Advanced Filtering with Custom Functions ---")
193
194 # Filter with custom function
195 def is_senior_employee(row):
196     return row['Experience'] > 10 and row['Salary'] > 70000
197
198 senior_employees = filter_df[filter_df.apply(is_senior_employee, axis=1)]
199 print(f"Senior employees (custom function): {len(senior_employees)}")
200
201 # Filter with lambda function

```

```

206 young_high_earners = filter_df[filter_df.apply(lambda x: x['Age']
207 print(f"Young high earners (lambda): {len(young_high_earners)}")
208
209 # Filter with multiple custom conditions
210 def complex_filter_func(row):
211     return (
212         row['Department'] in ['IT', 'Finance'] and
213         row['Salary'] > 60000 and
214         row['Experience'] > 5 and
215         row['Performance'] in ['Excellent', 'Good']
216     )
217
218
219 complex_filtered = filter_df[filter_df.apply(complex_filter_func,
220 print(f"Complex filtered employees: {len(complex_filtered)}")
221
222 # Using where() method
223 print(f"\n--- Using where() Method ---")
224
225 # where() with condition
226 salary_where = filter_df.where(filter_df['Salary'] > 70000)
227 print(f"where() method - high salary employees:\n{salary_where[['Name', 'Salary']]}")
228
229 # where() with multiple conditions
230 complex_where = filter_df.where(
231     (filter_df['Department'] == 'IT') & (filter_df['Salary'] > 60000)
232 )
233 print(f"where() method - IT high salary:\n{complex_where[['Name', 'Salary']]}")
234
235 # Using mask() method (opposite of where)
236 print(f"\n--- Using mask() Method ---")
237
238 # mask() with condition
239 salary_mask = filter_df.mask(filter_df['Salary'] <= 70000)
240 print(f"mask() method - high salary employees:\n{salary_mask[['Name', 'Salary']]}")
241
242 # Performance comparison
243 print(f"\n--- Performance Comparison ---")
244 import time
245
246 # Large dataset for performance testing
247 large_filter_data = {
248     'ID': range(1, 10001),
249     'Value': np.random.randn(10000),
250     'Category': np.random.choice(['A', 'B', 'C', 'D', 'E'], 10000),
251     'Score': np.random.randint(0, 100, 10000)
252 }
253
254 }
255 large_filter_df = pd.DataFrame(large_filter_data)
256
257 # Time different filtering methods
258 start_time = time.time()
259 result1 = large_filter_df[large_filter_df['Value'] > 0]
260 end_time = time.time()
261 print(f"Boolean indexing time: {end_time - start_time:.4f} second")
262
263 start_time = time.time()
264 result2 = large_filter_df.query('Value > 0')
265 end_time = time.time()
266 print(f"query() method time: {end_time - start_time:.4f} seconds")
267
268 start_time = time.time()
269

```

```

270 result3 = large_filter_df[large_filter_df.apply(lambda x: x['Valu
271 end_time = time.time()
272 print(f"apply() method time: {end_time - start_time:.4f} seconds"
273
274 print(f"All methods produced same result: {len(result1) == len(re
275
276 # Advanced query examples
277 print(f"\n--- Advanced Query Examples ---")
278
279 # Query with mathematical operations
280 math_query = filter_df.query('Salary * 1.1 > 70000')
281 print(f"Salary * 1.1 > 70000: {len(math_query)}")
282
283 # Query with string operations
284 string_query = filter_df.query('Name.str.len() > 10')
print(f"Name length > 10: {len(string_query)}")

# Query with date operations
date_query = filter_df.query('Start_Date.dt.year == 2020 and Star
print(f"Started in H2 2020: {len(date_query)}")

# Query with multiple conditions and parentheses
complex_query = filter_df.query(
    '(Department == "IT" or Department == "Finance") and '
    'Salary > 60000 and '
    'Experience > 5 and '
    'Performance in ["Excellent", "Good"]'
)
print(f"Complex query results: {len(complex_query)}")

print("\nAdvanced filtering and querying completed successfully!")

```

## 9. Multi-level Indexing

In [ ...

```

1 # Create data for multi-level indexing examples
2 np.random.seed(42)
3 multi_data = {
4     'Company': ['Apple', 'Apple', 'Apple', 'Apple', 'Google', 'Go
5     'Year': [2020, 2020, 2021, 2021, 2020, 2020, 2021, 2021, 2020
6     'Quarter': ['Q1', 'Q2', 'Q1', 'Q2', 'Q1', 'Q2', 'Q1', 'Q2', '
7     'Revenue': [58000, 62000, 65000, 68000, 45000, 48000, 52000,
8     'Profit': [12000, 14000, 15000, 16000, 8000, 9000, 10000, 110
9     'Employees': [150000, 155000, 160000, 165000, 120000, 125000,
10
11 }
12
13 multi_df = pd.DataFrame(multi_data)
14 print(f"Original DataFrame:\n{multi_df}")
15
16 # Create MultiIndex from columns
17 print(f"\n--- Creating MultiIndex from Columns ---")
18
19 # Set multiple columns as index
20 multi_indexed = multi_df.set_index(['Company', 'Year', 'Quarter'])
21 print(f"MultiIndexed DataFrame:\n{multi_indexed}")
22
23
24 # Create MultiIndex from tuples

```



```

25 print(f"\n--- Creating MultiIndex from Tuples ---")
26
27 # Create MultiIndex from tuples
28 tuples = [('Apple', 2020), ('Apple', 2021), ('Google', 2020), ('G
29 multi_index = pd.MultiIndex.from_tuples(tuples, names=['Company',
30 print(f"MultiIndex from tuples:\n{multi_index}")
31
32 # Create DataFrame with MultiIndex
33 multi_df_tuples = pd.DataFrame({
34     'Q1_Revenue': [58000, 65000, 45000, 52000, 38000, 44000],
35     'Q2_Revenue': [62000, 68000, 48000, 55000, 41000, 47000],
36     'Q1_Profit': [12000, 15000, 8000, 10000, 6000, 8000],
37     'Q2_Profit': [14000, 16000, 9000, 11000, 7000, 9000]
38 }, index=multi_index)
39 print(f"DataFrame with MultiIndex from tuples:\n{multi_df_tuples}")
40
41
42 # Create MultiIndex from arrays
43 print(f"\n--- Creating MultiIndex from Arrays ---")
44
45 # Create MultiIndex from arrays
46 companies = ['Apple', 'Apple', 'Google', 'Google', 'Microsoft', '
47 years = [2020, 2021, 2020, 2021, 2020, 2021]
48 multi_index_arrays = pd.MultiIndex.from_arrays([companies, years])
49 print(f"MultiIndex from arrays:\n{multi_index_arrays}")
50
51 # Create MultiIndex from product
52 print(f"\n--- Creating MultiIndex from Product ---")
53
54 # Create MultiIndex from product
55 companies = ['Apple', 'Google', 'Microsoft']
56 years = [2020, 2021]
57 quarters = ['Q1', 'Q2']
58 multi_index_product = pd.MultiIndex.from_product([companies, year
59 print(f"MultiIndex from product:\n{multi_index_product}")
60
61 # Indexing with MultiIndex
62 print(f"\n--- Indexing with MultiIndex ---")
63
64 # Access by first level
65 apple_data = multi_indexed.loc['Apple']
66 print(f"Apple data:\n{apple_data}")
67
68 # Access by multiple levels
69 apple_2020 = multi_indexed.loc[('Apple', 2020)]
70 print(f"Apple 2020 data:\n{apple_2020}")
71
72 # Access by all levels
73 apple_2020_q1 = multi_indexed.loc[('Apple', 2020, 'Q1')]
74 print(f"Apple 2020 Q1 data:\n{apple_2020_q1}")
75
76 # Slicing with MultiIndex
77 print(f"\n--- Slicing with MultiIndex ---")
78
79 # Slice by first level
80 apple_google = multi_indexed.loc[['Apple', 'Google']]
81 print(f"Apple and Google data:\n{apple_google}")
82
83 # Slice by multiple levels
84 apple_2020_2021 = multi_indexed.loc[('Apple', [2020, 2021])]
85 print(f"Apple 2020-2021 data:\n{apple_2020_2021}")
86
87
88

```



```

89
90 # Slice by all levels
91 apple_2020_q1_q2 = multi_indexed.loc(['Apple', 2020, ['Q1', 'Q2']]
92 print(f"Apple 2020 Q1-Q2 data:\n{apple_2020_q1_q2}")
93
94 # Cross-section selection
95 print(f"\n--- Cross-section Selection ---")
96
97 # Select all data for 2020
98 data_2020 = multi_indexed.xs(2020, level='Year')
99 print(f"All data for 2020:\n{data_2020}")
100
101 # Select all data for Q1
102 data_q1 = multi_indexed.xs('Q1', level='Quarter')
103 print(f"All data for Q1:\n{data_q1}")
104
105 # Select Apple data for Q1
106 apple_q1 = multi_indexed.xs(['Apple', 'Q1'], level=['Company', 'Q
107 print(f"Apple data for Q1:\n{apple_q1}")
108
109 # Select specific company and year
110 apple_2020_xs = multi_indexed.xs(['Apple', 2020], level=['Company
111 print(f"Apple 2020 data (xs):\n{apple_2020_xs}")
112
113 # MultiIndex operations
114 print(f"\n--- MultiIndex Operations ---")
115
116 # Sort by index
117 sorted_multi = multi_indexed.sort_index()
118 print(f"Sorted by index:\n{sorted_multi}")
119
120 # Sort by specific level
121 sorted_by_year = multi_indexed.sort_index(level='Year')
122 print(f"Sorted by Year:\n{sorted_by_year}")
123
124 # Sort by multiple levels
125 sorted_by_company_year = multi_indexed.sort_index(level=['Company
126 print(f"Sorted by Company and Year:\n{sorted_by_company_year}")
127
128 # Swap levels
129 swapped_levels = multi_indexed.swaplevel('Year', 'Quarter')
130 print(f"Swapped levels (Year, Quarter):\n{swapped_levels}")
131
132 # Reorder levels
133 reordered_levels = multi_indexed.reorder_levels(['Year', 'Company
134 print(f"Reordered levels (Year, Company, Quarter):\n{reordered_le
135
136 # Drop levels
137 dropped_level = multi_indexed.droplevel('Quarter')
138 print(f"Dropped Quarter level:\n{dropped_level}")
139
140 # Groupby with MultiIndex
141 print(f"\n--- Groupby with MultiIndex ---")
142
143 # Group by first level
144 grouped_company = multi_indexed.groupby(level='Company').sum()
145 print(f"Grouped by Company:\n{grouped_company}")
146
147 # Group by multiple levels
148 grouped_company_year = multi_indexed.groupby(level=['Company', 'Y

```

```

153 print(f"Grouped by Company and Year:\n{grouped_company_year}")
154
155 # Group by specific level
156 grouped_year = multi_indexed.groupby(level='Year').mean()
157 print(f"Grouped by Year (mean):\n{grouped_year}")
158
159 # Aggregation with MultiIndex
160 print(f"\n--- Aggregation with MultiIndex ---")
161
162 # Multiple aggregations
163 agg_results = multi_indexed.groupby(level='Company').agg({
164     'Revenue': ['sum', 'mean', 'std'],
165     'Profit': ['sum', 'mean'],
166     'Employees': 'last'
167 })
168
169 print(f"Aggregation results:\n{agg_results}")
170
171 # Custom aggregation
172 def revenue_growth(series):
173     return series.iloc[-1] - series.iloc[0]
174
175 growth_results = multi_indexed.groupby(level='Company')['Revenue']
176 print(f"Revenue growth by company:\n{growth_results}")
177
178 # Pivot with MultiIndex
179 print(f"\n--- Pivot with MultiIndex ---")
180
181 # Reset index to use pivot
182 pivot_df = multi_indexed.reset_index()
183 print(f"Reset index for pivot:\n{pivot_df}")
184
185 # Pivot to create MultiIndex columns
186 pivoted = pivot_df.pivot(index=['Company', 'Year'], columns='Quarter')
187 print(f"Pivoted with MultiIndex columns:\n{pivoted}")
188
189 # Pivot with multiple values
190 pivoted_multi = pivot_df.pivot(index=['Company', 'Year'], columns=['Quarter', 'Revenue'])
191 print(f"Pivoted with multiple values:\n{pivoted_multi}")
192
193 # Stack and unstack with MultiIndex
194 print(f"\n--- Stack and Unstack with MultiIndex ---")
195
196 # Unstack (convert index to columns)
197 unstacked = multi_indexed.unstack('Quarter')
198 print(f"Unstacked by Quarter:\n{unstacked}")
199
200 # Unstack multiple levels
201 unstacked_multi = multi_indexed.unstack(['Year', 'Quarter'])
202 print(f"Unstacked by Year and Quarter:\n{unstacked_multi}")
203
204 # Stack (convert columns to index)
205 stacked = unstacked.stack('Quarter')
206 print(f"Stacked back:\n{stacked}")
207
208 # Melt with MultiIndex
209 print(f"\n--- Melt with MultiIndex ---")
210
211 # Melt the pivoted data
212 melted = pivoted.reset_index().melt(
213     id_vars=['Company', 'Year'],
214     value_vars=['Quarter', 'Revenue'],
215     var_name='Variable',
216     value_name='Value')

```

```

217         value_vars=['Q1', 'Q2'],
218         var_name='Quarter',
219         value_name='Revenue'
220     )
221     print(f"Melted data:\n{melted}")
222
223     # Set MultiIndex after melt
224     melted_multi = melted.set_index(['Company', 'Year', 'Quarter'])
225     print(f"Melted with MultiIndex:\n{melted_multi}")
226
227     # MultiIndex with different data types
228     print(f"\n--- MultiIndex with Different Data Types ---")
229
230     # Create MultiIndex with different data types
231     mixed_data = {
232         'Category': ['Electronics', 'Electronics', 'Clothing', 'Clothing', 'Books', 'Books'],
233         'Subcategory': ['Phones', 'Laptops', 'Shirts', 'Pants', 'Fiction', 'Non-Fiction'],
234         'Price': [800, 1200, 50, 80, 15, 25],
235         'Stock': [100, 50, 200, 150, 300, 250]
236     }
237
238     mixed_df = pd.DataFrame(mixed_data)
239     mixed_multi = mixed_df.set_index(['Category', 'Subcategory'])
240     print(f"Mixed data with MultiIndex:\n{mixed_multi}")
241
242     # Access mixed data
243     electronics = mixed_multi.loc['Electronics']
244     print(f"Electronics data:\n{electronics}")
245
246     phones = mixed_multi.loc[('Electronics', 'Phones')]
247     print(f"Phones data:\n{phones}")
248
249     # MultiIndex with time series
250     print(f"\n--- MultiIndex with Time Series ---")
251
252     # Create time series data with MultiIndex
253     dates = pd.date_range('2024-01-01', periods=12, freq='M')
254     time_data = {
255         'Company': ['Apple'] * 12 + ['Google'] * 12,
256         'Date': list(dates) * 2,
257         'Revenue': np.random.randint(50000, 80000, 24),
258         'Profit': np.random.randint(10000, 20000, 24)
259     }
260
261     time_df = pd.DataFrame(time_data)
262     time_multi = time_df.set_index(['Company', 'Date'])
263     print(f"Time series with MultiIndex:\n{time_multi.head(10)}")
264
265     # Time-based operations with MultiIndex
266     time_grouped = time_multi.groupby(level='Company').resample('Q',
267     print(f"Quarterly data by company:\n{time_grouped}")
268
269     # MultiIndex performance
270     print(f"\n--- MultiIndex Performance ---")
271     import time
272
273     # Create large MultiIndex dataset
274     large_companies = ['Company_' + str(i) for i in range(100)]
275     large_years = [2020, 2021, 2022, 2023]
276     large_quarters = ['Q1', 'Q2', 'Q3', 'Q4']

```

```

281
282 large_multi_index = pd.MultiIndex.from_product(
283     [large_companies, large_years, large_quarters],
284     names=['Company', 'Year', 'Quarter']
285 )
286
287 large_data = pd.DataFrame({
288     'Revenue': np.random.randint(10000, 100000, len(large_multi_index)),
289     'Profit': np.random.randint(1000, 10000, len(large_multi_index))
290 }, index=large_multi_index)
291
292 print(f"Large MultiIndex dataset shape: {large_data.shape}")
293
294 # Time different operations
295 start_time = time.time()
296 result1 = large_data.loc['Company_0']
297 end_time = time.time()
298 print(f"Access by first level: {end_time - start_time:.4f} seconds")
299
300 start_time = time.time()
301 result2 = large_data.xs(2020, level='Year')
302 end_time = time.time()
303 print(f"Cross-section selection: {end_time - start_time:.4f} seconds")
304
305 start_time = time.time()
306 result3 = large_data.groupby(level='Company').sum()
307 end_time = time.time()
308 print(f"Groupby operation: {end_time - start_time:.4f} seconds")
309
310 # MultiIndex with missing data
311 print(f"\n--- MultiIndex with Missing Data ---")
312
313 # Create data with missing values
314 missing_data = {
315     'Company': ['Apple', 'Apple', 'Google', 'Google', 'Microsoft'],
316     'Year': [2020, 2021, 2020, 2021, 2020],
317     'Revenue': [58000, 65000, 45000, 52000, 38000],
318     'Profit': [12000, 15000, 8000, 11000, 6000]
319 }
320
321 missing_df = pd.DataFrame(missing_data)
322 missing_multi = missing_df.set_index(['Company', 'Year'])
323 print(f"Data with missing combinations:\n{missing_multi}")
324
325 # Fill missing data
326 filled_data = missing_multi.reindex(
327     pd.MultiIndex.from_product(['Apple', 'Google', 'Microsoft'],
328                                 ['2020', '2021']),
329     fill_value=0
330 )
331 print(f"Filled missing data:\n{filled_data}")
332
333 # Interpolate missing data
334 interpolated_data = missing_multi.reindex(
335     pd.MultiIndex.from_product(['Apple', 'Google', 'Microsoft'],
336                                 ['2020', '2021']),
337     ).interpolate()
338 print(f"Interpolated missing data:\n{interpolated_data}")
339
340 print("\nMulti-level indexing completed successfully!")

```

## 10. Performance Optimization Techniques

In [ ...

```
1  # Performance optimization techniques for pandas
2
3  import time
4  import memory_profiler
5  import pandas as pd
6  import numpy as np
7
8  # Create large dataset for performance testing
9  print("Creating large dataset for performance testing...")
10 np.random.seed(42)
11 n_rows = 100000
12 n_cols = 10
13
14 perf_data = {
15     'ID': range(n_rows),
16     'Category': np.random.choice(['A', 'B', 'C', 'D', 'E'], n_rows),
17     'Value1': np.random.randn(n_rows),
18     'Value2': np.random.randn(n_rows),
19     'Value3': np.random.randn(n_rows),
20     'Value4': np.random.randn(n_rows),
21     'Value5': np.random.randn(n_rows),
22     'Value6': np.random.randn(n_rows),
23     'Value7': np.random.randn(n_rows),
24     'Value8': np.random.randn(n_rows),
25     'Value9': np.random.randn(n_rows),
26     'Value10': np.random.randn(n_rows),
27     'Date': pd.date_range('2020-01-01', periods=n_rows, freq='H')
28 }
29
30
31 perf_df = pd.DataFrame(perf_data)
32 print(f"Dataset created: {perf_df.shape}")
33 print(f"Memory usage: {perf_df.memory_usage(deep=True).sum() / 1024} MB")
34
35 # 1. Data Types Optimization
36 print(f"\n--- 1. Data Types Optimization ---")
37
38 # Check current data types
39 print(f"Current data types:\n{perf_df.dtypes}")
40
41 # Optimize data types
42 def optimize_dtypes(df):
43     df_optimized = df.copy()
44
45     # Convert integer columns to appropriate types
46     for col in df_optimized.select_dtypes(include=['int64']).columns:
47         if df_optimized[col].min() >= 0:
48             if df_optimized[col].max() < 255:
49                 df_optimized[col] = df_optimized[col].astype('uint8')
50             elif df_optimized[col].max() < 65535:
51                 df_optimized[col] = df_optimized[col].astype('uint16')
52             elif df_optimized[col].max() < 4294967295:
53                 df_optimized[col] = df_optimized[col].astype('uint32')
54             else:
55                 df_optimized[col] = df_optimized[col].astype('uint64')
56         elif df_optimized[col].min() > -128 and df_optimized[col].max() < 128:
57             df_optimized[col] = df_optimized[col].astype('int8')
58         elif df_optimized[col].min() > -2147483648 and df_optimized[col].max() < 2147483647:
59             df_optimized[col] = df_optimized[col].astype('int16')
60         elif df_optimized[col].min() > -9223372036854775808 and df_optimized[col].max() < 9223372036854775807:
61             df_optimized[col] = df_optimized[col].astype('int32')
62         else:
63             df_optimized[col] = df_optimized[col].astype('int64')
```

```

59         elif df_optimized[col].min() > -32768 and df_optimize
60             df_optimized[col] = df_optimized[col].astype('int
61         elif df_optimized[col].min() > -2147483648 and df_opt
62             df_optimized[col] = df_optimized[col].astype('int
63
64     # Convert float columns to appropriate types
65     for col in df_optimized.select_dtypes(include=['float64']).co
66         df_optimized[col] = df_optimized[col].astype('float32')
67
68     # Convert object columns to category if they have few unique
69     for col in df_optimized.select_dtypes(include=['object']).col
70         if df_optimized[col].nunique() / len(df_optimized) < 0.5:
71             df_optimized[col] = df_optimized[col].astype('categor
72
73     return df_optimized
74
75 # Apply optimization
76 perf_df_optimized = optimize_dtypes(perf_df)
77
78 print(f"Optimized data types:\n{perf_df_optimized.dtypes}")
79 print(f"Memory usage before: {perf_df.memory_usage(deep=True).sum
80 print(f"Memory usage after: {perf_df_optimized.memory_usage(deep=
81 print(f"Memory reduction: {(1 - perf_df_optimized.memory_usage(de
82
83 # 2. Vectorized Operations
84 print(f"\n--- 2. Vectorized Operations ---")
85
86 # Slow: Using apply with lambda
87 start_time = time.time()
88 slow_result = perf_df.apply(lambda x: x['Value1'] + x['Value2'],
89 end_time = time.time()
90 print(f"Slow method (apply): {end_time - start_time:.4f} seconds"
91
92 # Fast: Using vectorized operations
93 start_time = time.time()
94 fast_result = perf_df['Value1'] + perf_df['Value2']
95 end_time = time.time()
96 print(f"Fast method (vectorized): {end_time - start_time:.4f} sec
97 print(f"Speed improvement: {(end_time - start_time) / (end_time -
98
99 # More vectorized operations examples
100 print(f"\n--- Vectorized Operations Examples ---")
101
102 # Slow: Using apply for complex calculations
103 start_time = time.time()
104 slow_complex = perf_df.apply(lambda x: np.sqrt(x['Value1']**2 + x
105 end_time = time.time()
106 print(f"Slow complex calculation: {end_time - start_time:.4f} sec
107
108 # Fast: Using vectorized operations
109 start_time = time.time()
110 fast_complex = np.sqrt(perf_df['Value1']**2 + perf_df['Value2']**
111 end_time = time.time()
112 print(f"Fast complex calculation: {end_time - start_time:.4f} sec
113
114 # 3. Efficient Filtering
115 print(f"\n--- 3. Efficient Filtering ---")
116
117 # Slow: Using apply for filtering
118 start_time = time.time()
119

```

```

123 slow_filter = perf_df[perf_df.apply(lambda x: x['Value1'] > 0 and
124 end_time = time.time()
125 print(f"Slow filtering: {end_time - start_time:.4f} seconds")
126
127 # Fast: Using boolean indexing
128 start_time = time.time()
129 fast_filter = perf_df[(perf_df['Value1'] > 0) & (perf_df['Value2']
130 end_time = time.time()
131 print(f"Fast filtering: {end_time - start_time:.4f} seconds")
132
133 # 4. Efficient GroupBy Operations
134 print(f"\n--- 4. Efficient GroupBy Operations ---")
135
136 # Slow: Using apply with custom function
137 def slow_groupby_func(group):
138     return group['Value1'].sum() + group['Value2'].sum()
139
140
141 start_time = time.time()
142 slow_groupby = perf_df.groupby('Category').apply(slow_groupby_func)
143 end_time = time.time()
144 print(f"Slow groupby: {end_time - start_time:.4f} seconds")
145
146 # Fast: Using built-in aggregation
147 start_time = time.time()
148 fast_groupby = perf_df.groupby('Category')[['Value1', 'Value2']].
149 end_time = time.time()
150 print(f"Fast groupby: {end_time - start_time:.4f} seconds")
151
152 # 5. Memory-Efficient Operations
153 print(f"\n--- 5. Memory-Efficient Operations ---")
154
155 # Chunked processing for large datasets
156 def process_chunks(df, chunk_size=10000):
157     results = []
158     for chunk in pd.read_csv(StringIO(df.to_csv()), chunksize=chu
159         # Process each chunk
160         chunk_result = chunk.groupby('Category').sum()
161         results.append(chunk_result)
162     return pd.concat(results).groupby(level=0).sum()
163
164
165 # In-place operations to save memory
166 print(f"Memory before in-place operations: {perf_df.memory_usage(
167
168 # In-place operations
169 perf_df['Value1'] = perf_df['Value1'] * 2
170 perf_df['Value2'] = perf_df['Value2'] + 1
171
172
173 print(f"Memory after in-place operations: {perf_df.memory_usage(d
174
175 # 6. Categorical Data Optimization
176 print(f"\n--- 6. Categorical Data Optimization ---")
177
178 # Create data with repeated strings
179 string_data = {
180     'ID': range(10000),
181     'Category': np.random.choice(['Category_A', 'Category_B', 'Ca
182     'Status': np.random.choice(['Active', 'Inactive', 'Pending',
183     'Value': np.random.randn(10000)
184 }
185
186

```



```

187 string_df = pd.DataFrame(string_data)
188 print(f"String data memory usage: {string_df.memory_usage(deep=True).sum()}")
189
190 # Convert to categorical
191 string_df['Category'] = string_df['Category'].astype('category')
192 string_df['Status'] = string_df['Status'].astype('category')
193
194 print(f"Categorical data memory usage: {string_df.memory_usage(deep=True).sum()}")
195 print(f"Memory reduction: {(1 - string_df.memory_usage(deep=True).sum() / string_df.memory_usage(deep=True).sum()) * 100}%")
196
197 # 7. Efficient String Operations
198 print(f"\n--- 7. Efficient String Operations ---")
199
200 # Create string data
201 string_data = {
202     'Name': [f'Employee_{i}' for i in range(10000)],
203     'Email': [f'employee_{i}@company.com' for i in range(10000)],
204     'Department': np.random.choice(['IT', 'HR', 'Finance', 'Marketing'], 10000)
205 }
206
207 string_df = pd.DataFrame(string_data)
208
209 # Slow: Using apply with string methods
210 start_time = time.time()
211 slow_string = string_df['Name'].apply(lambda x: x.upper())
212 end_time = time.time()
213 print(f"Slow string operations: {end_time - start_time:.4f} seconds")
214
215 # Fast: Using vectorized string methods
216 start_time = time.time()
217 fast_string = string_df['Name'].str.upper()
218 end_time = time.time()
219 print(f"Fast string operations: {end_time - start_time:.4f} seconds")
220
221 # 8. Efficient Date Operations
222 print(f"\n--- 8. Efficient Date Operations ---")
223
224 # Create date data
225 date_data = {
226     'Date': pd.date_range('2020-01-01', periods=10000, freq='H'),
227     'Value': np.random.randn(10000)
228 }
229
230 date_df = pd.DataFrame(date_data)
231
232 # Slow: Using apply for date operations
233 start_time = time.time()
234 slow_date = date_df['Date'].apply(lambda x: x.year)
235 end_time = time.time()
236 print(f"Slow date operations: {end_time - start_time:.4f} seconds")
237
238 # Fast: Using vectorized date operations
239 start_time = time.time()
240 fast_date = date_df['Date'].dt.year
241 end_time = time.time()
242 print(f"Fast date operations: {end_time - start_time:.4f} seconds")
243
244 # 9. Efficient Concatenation
245 print(f"\n--- 9. Efficient Concatenation ---")

```

```

251 # Create multiple DataFrames
252 dfs = [pd.DataFrame({'A': np.random.randn(1000), 'B': np.random.r
253
254 # Slow: Using append in loop
255 start_time = time.time()
256 slow_concat = pd.DataFrame()
257 for df in dfs:
258     slow_concat = slow_concat.append(df, ignore_index=True)
259 end_time = time.time()
260 print(f"Slow concatenation: {end_time - start_time:.4f} seconds")
261
262 # Fast: Using pd.concat
263 start_time = time.time()
264 fast_concat = pd.concat(dfs, ignore_index=True)
265 end_time = time.time()
266 print(f"Fast concatenation: {end_time - start_time:.4f} seconds")
267
268
269 # 10. Efficient Merging
270 print(f"\n--- 10. Efficient Merging ---")
271
272 # Create DataFrames for merging
273 df1 = pd.DataFrame({
274     'ID': range(10000),
275     'Value1': np.random.randn(10000)
276 })
277
278 df2 = pd.DataFrame({
279     'ID': range(10000),
280     'Value2': np.random.randn(10000)
281 })
282
283
284 # Slow: Using merge with non-index columns
285 start_time = time.time()
286 slow_merge = pd.merge(df1, df2, on='ID')
287 end_time = time.time()
288 print(f"Slow merge: {end_time - start_time:.4f} seconds")
289
290 # Fast: Using merge with indexed columns
291 df1_indexed = df1.set_index('ID')
292 df2_indexed = df2.set_index('ID')
293
294 start_time = time.time()
295 fast_merge = df1_indexed.join(df2_indexed)
296 end_time = time.time()
297 print(f"Fast merge (indexed): {end_time - start_time:.4f} seconds")
298
299
300 # 11. Memory Profiling
301 print(f"\n--- 11. Memory Profiling ---")
302
303 # Function to profile memory usage
304 def memory_intensive_operation(df):
305     # Create a copy
306     df_copy = df.copy()
307
308     # Add new columns
309     df_copy['New_Col1'] = df_copy['Value1'] * 2
310     df_copy['New_Col2'] = df_copy['Value2'] + 1
311
312     # Group by and aggregate
313     result = df_copy.groupby('Category').sum()
314

```

```

315
316         return result
317
318     # Profile memory usage
319     import tracemalloc
320     tracemalloc.start()
321
322     # Run memory-intensive operation
323     result = memory_intensive_operation(perf_df)
324
325     # Get memory usage
326     current, peak = tracemalloc.get_traced_memory()
327     print(f"Current memory usage: {current / 1024**2:.2f} MB")
328     print(f"Peak memory usage: {peak / 1024**2:.2f} MB")
329
330     tracemalloc.stop()
331
332
333     # 12. Performance Monitoring
334     print(f"\n--- 12. Performance Monitoring ---")
335
336     # Function to time operations
337     def time_operation(func, *args, **kwargs):
338         start_time = time.time()
339         result = func(*args, **kwargs)
340         end_time = time.time()
341         return result, end_time - start_time
342
343     # Time different operations
344     def operation1(df):
345         return df.groupby('Category').sum()
346
347     def operation2(df):
348         return df[df['Value1'] > 0]
349
350     def operation3(df):
351         return df.apply(lambda x: x['Value1'] + x['Value2'], axis=1)
352
353     # Time operations
354     result1, time1 = time_operation(operation1, perf_df)
355     result2, time2 = time_operation(operation2, perf_df)
356     result3, time3 = time_operation(operation3, perf_df)
357
358     print(f"Operation 1 (groupby): {time1:.4f} seconds")
359     print(f"Operation 2 (filtering): {time2:.4f} seconds")
360     print(f"Operation 3 (apply): {time3:.4f} seconds")
361
362
363     # 13. Best Practices Summary
364     print(f"\n--- 13. Best Practices Summary ---")
365
366     print("""
367     Performance Optimization Best Practices:
368
369     1. Data Types:
370         - Use appropriate data types (int8, int16, float32, category)
371         - Convert object columns to category when possible
372         - Use nullable integer types for missing values
373
374     2. Operations:
375         - Use vectorized operations instead of apply()
376         - Use boolean indexing instead of apply() for filtering
377
378

```

```

379         - Use built-in aggregation functions instead of custom functions
380
381     3. Memory:
382         - Use in-place operations when possible
383         - Process data in chunks for large datasets
384         - Use categorical data for repeated strings
385
386     4. String Operations:
387         - Use .str accessor for vectorized string operations
388         - Avoid apply() with string methods
389
390     5. Date Operations:
391         - Use .dt accessor for vectorized date operations
392         - Avoid apply() with date methods
393
394     6. Concatenation:
395         - Use pd.concat() instead of append() in loops
396         - Pre-allocate arrays when possible
397
398     7. Merging:
399         - Use indexed columns for joining
400         - Use appropriate join types
401
402     8. Monitoring:
403         - Profile memory usage
404         - Time critical operations
405         - Use appropriate data structures
406 """
407
408 print("\nPerformance optimization techniques completed successfully")

```

## 11. Exporting and Importing Data

In [ ...

```

1  # Create sample data for export/import examples
2  import pandas as pd
3  import numpy as np
4  import os
5  import json
6  import pickle
7  import sqlite3
8  from io import StringIO
9
10
11  # Create comprehensive sample data
12  export_data = {
13      'Employee_ID': range(1, 101),
14      'Name': [f'Employee_{i}' for i in range(1, 101)],
15      'Department': np.random.choice(['IT', 'HR', 'Finance', 'Marketing'], 100),
16      'Salary': np.random.normal(60000, 15000, 100).astype(int),
17      'Experience': np.random.randint(1, 25, 100),
18      'Performance': np.random.choice(['Excellent', 'Good', 'Average', 'Poor'], 100),
19      'Location': np.random.choice(['New York', 'Los Angeles', 'Chicago'], 100),
20      'Age': np.random.randint(22, 65, 100),
21      'Bonus': np.random.normal(5000, 2000, 100).astype(int),
22      'Start_Date': pd.date_range('2020-01-01', periods=100, freq='D'),
23      'Is_Active': np.random.choice([True, False], 100)
24  }
25

```

```

26 export_df = pd.DataFrame(export_data)
27 print(f"Sample DataFrame for export/import:\n{export_df.head()}")
28 print(f"DataFrame shape: {export_df.shape}")
29
30
31 # 1. CSV Export and Import
32 print(f"\n--- 1. CSV Export and Import ----")
33
34 # Export to CSV
35 export_df.to_csv('sample_data.csv', index=False)
36 print("Data exported to CSV: sample_data.csv")
37
38 # Import from CSV
39 imported_csv = pd.read_csv('sample_data.csv')
40 print(f"Data imported from CSV:\n{imported_csv.head()}")
41 print(f"Shape: {imported_csv.shape}")
42
43 # CSV with different parameters
44 print(f"\n--- CSV with Different Parameters ----")
45
46 # Export with custom separator
47 export_df.to_csv('sample_data_semicolon.csv', sep=';', index=False)
48 print("Data exported with semicolon separator")
49
50 # Import with custom separator
51 imported_semicolon = pd.read_csv('sample_data_semicolon.csv', sep=';')
52 print(f"Data imported with semicolon separator:\n{imported_semicolon.head()}")
53
54 # Export with specific columns
55 export_df[['Name', 'Department', 'Salary']].to_csv('sample_data_subset.csv', index=False)
56 print("Subset data exported to CSV")
57
58 # Import with specific columns
59 imported_subset = pd.read_csv('sample_data_subset.csv')
60 print(f"Subset data imported:\n{imported_subset.head()}")
61
62 # 2. Excel Export and Import
63 print(f"\n--- 2. Excel Export and Import ----")
64
65 # Export to Excel
66 export_df.to_excel('sample_data.xlsx', index=False, sheet_name='Export')
67 print("Data exported to Excel: sample_data.xlsx")
68
69 # Import from Excel
70 imported_excel = pd.read_excel('sample_data.xlsx', sheet_name='Export')
71 print(f"Data imported from Excel:\n{imported_excel.head()}")
72
73 # Excel with multiple sheets
74 print(f"\n--- Excel with Multiple Sheets ----")
75
76 # Create multiple DataFrames
77 dept_summary = export_df.groupby('Department').agg({
78     'Salary': ['mean', 'std', 'count'],
79     'Experience': 'mean'
80 }).round(2)
81
82 performance_summary = export_df.groupby('Performance').size().reset_index()
83
84 # Export to multiple sheets
85 with pd.ExcelWriter('sample_data_multi.xlsx') as writer:

```

```

90     export_df.to_excel(writer, sheet_name='Employees', index=False)
91     dept_summary.to_excel(writer, sheet_name='Department_Summary')
92     performance_summary.to_excel(writer, sheet_name='Performance_Summary')
93
94     print("Data exported to multiple Excel sheets")
95
96     # Import from multiple sheets
97     imported_multi = pd.read_excel('sample_data_multi.xlsx', sheet_name='Employees')
98     print(f>Data imported from multiple sheets:\n{imported_multi['Employees'].head()})
99     print(f>Department summary:\n{imported_multi['Department_Summary'].head()})
100
101     # 3. JSON Export and Import
102     print(f">\n--- 3. JSON Export and Import ---")
103
104     # Export to JSON
105     export_df.to_json('sample_data.json', orient='records', indent=2)
106     print("Data exported to JSON: sample_data.json")
107
108     # Import from JSON
109     imported_json = pd.read_json('sample_data.json')
110     print(f>Data imported from JSON:\n{imported_json.head()})
111
112     # JSON with different orientations
113     print(f">\n--- JSON with Different Orientations ---")
114
115     # Export with different orientations
116     export_df.to_json('sample_data_index.json', orient='index', indent=2)
117     export_df.to_json('sample_data_values.json', orient='values', indent=2)
118     export_df.to_json('sample_data_columns.json', orient='columns', indent=2)
119
120     print("Data exported with different JSON orientations")
121
122     # Import with different orientations
123     imported_index = pd.read_json('sample_data_index.json', orient='index')
124     imported_values = pd.read_json('sample_data_values.json', orient='values')
125     imported_columns = pd.read_json('sample_data_columns.json', orient='columns')
126
127     print(f">Index orientation:\n{imported_index.head()})")
128     print(f">Values orientation:\n{imported_values.head()})")
129     print(f">Columns orientation:\n{imported_columns.head()})")
130
131     # 4. Parquet Export and Import
132     print(f">\n--- 4. Parquet Export and Import ---")
133
134     # Export to Parquet
135     export_df.to_parquet('sample_data.parquet', index=False)
136     print("Data exported to Parquet: sample_data.parquet")
137
138     # Import from Parquet
139     imported_parquet = pd.read_parquet('sample_data.parquet')
140     print(f>Data imported from Parquet:\n{imported_parquet.head()})")
141
142     # 5. Pickle Export and Import
143     print(f">\n--- 5. Pickle Export and Import ---")
144
145     # Export to Pickle
146     export_df.to_pickle('sample_data.pkl')
147     print("Data exported to Pickle: sample_data.pkl")
148
149     # Import from Pickle
150     imported_pickle = pd.read_pickle('sample_data.pkl')
151     print(f>Data imported from Pickle:\n{imported_pickle.head()})")
152
153

```

```

154 imported_pickle = pd.read_pickle('sample_data.pkl')
155 print(f"Data imported from Pickle:\n{imported_pickle.head()}")
156
157 # 6. SQL Database Export and Import
158 print(f"\n--- 6. SQL Database Export and Import ---")
159
160 # Create SQLite database
161 conn = sqlite3.connect('sample_data.db')
162 print("SQLite database created: sample_data.db")
163
164 # Export to SQL
165 export_df.to_sql('employees', conn, if_exists='replace', index=False)
166 print("Data exported to SQL table: employees")
167
168 # Import from SQL
169 imported_sql = pd.read_sql('SELECT * FROM employees', conn)
170 print(f"Data imported from SQL:\n{imported_sql.head()}")
171
172 # SQL with queries
173 print(f"\n--- SQL with Queries ---")
174
175 # Complex SQL query
176 query = """
177 SELECT Department,
178         COUNT(*) as Employee_Count,
179         AVG(Salary) as Avg_Salary,
180         MAX(Salary) as Max_Salary,
181         MIN(Salary) as Min_Salary
182 FROM employees
183 GROUP BY Department
184 ORDER BY Avg_Salary DESC
185 """
186
187 imported_query = pd.read_sql(query, conn)
188 print(f"SQL query result:\n{imported_query}")
189
190 # Close database connection
191 conn.close()
192
193 # 7. HDF5 Export and Import
194 print(f"\n--- 7. HDF5 Export and Import ---")
195
196 # Export to HDF5
197 export_df.to_hdf('sample_data.h5', key='employees', mode='w')
198 print("Data exported to HDF5: sample_data.h5")
199
200 # Import from HDF5
201 imported_hdf5 = pd.read_hdf('sample_data.h5', key='employees')
202 print(f"Data imported from HDF5:\n{imported_hdf5.head()}")
203
204 # HDF5 with multiple datasets
205 print(f"\n--- HDF5 with Multiple Datasets ---")
206
207 # Export multiple datasets
208 with pd.HDFStore('sample_data_multi.h5', mode='w') as store:
209     store['employees'] = export_df
210     store['dept_summary'] = dept_summary
211     store['performance_summary'] = performance_summary
212
213 print("Multiple datasets exported to HDF5")
214

```



```

218
219 # Import multiple datasets
220 with pd.HDFStore('sample_data_multi.h5', mode='r') as store:
221     imported_employees = store['employees']
222     imported_dept = store['dept_summary']
223     imported_perf = store['performance_summary']
224
225 print(f"Multiple datasets imported from HDF5")
226 print(f"Employees: {imported_employees.shape}")
227 print(f"Department summary: {imported_dept.shape}")
228 print(f"Performance summary: {imported_perf.shape}")
229
230
231 # 8. Feather Export and Import
232 print(f"\n--- 8. Feather Export and Import ---")
233
234 # Export to Feather
235 export_df.to_feather('sample_data.feather')
236 print("Data exported to Feather: sample_data.feather")
237
238 # Import from Feather
239 imported_feather = pd.read_feather('sample_data.feather')
240 print(f"Data imported from Feather:\n{imported_feather.head()}")
241
242 # 9. Stata Export and Import
243 print(f"\n--- 9. Stata Export and Import ---")
244
245 # Export to Stata
246 export_df.to_stata('sample_data.dta', write_index=False)
247 print("Data exported to Stata: sample_data.dta")
248
249 # Import from Stata
250 imported_stata = pd.read_stata('sample_data.dta')
251 print(f"Data imported from Stata:\n{imported_stata.head()}")
252
253
254 # 10. Clipboard Export and Import
255 print(f"\n--- 10. Clipboard Export and Import ---")
256
257 # Export to clipboard
258 export_df.head(10).to_clipboard(index=False)
259 print("Data exported to clipboard (first 10 rows)")
260
261 # Import from clipboard
262 imported_clipboard = pd.read_clipboard()
263 print(f"Data imported from clipboard:\n{imported_clipboard.head()}")
264
265
266 # 11. StringIO Export and Import
267 print(f"\n--- 11. StringIO Export and Import ---")
268
269 # Export to StringIO
270 csv_string = export_df.to_csv(index=False)
271 print("Data exported to StringIO")
272
273 # Import from StringIO
274 imported_stringio = pd.read_csv(StringIO(csv_string))
275 print(f"Data imported from StringIO:\n{imported_stringio.head()}")
276
277
278 # 12. Custom Export Functions
279 print(f"\n--- 12. Custom Export Functions ---")
280
281 # Custom export function

```

```

282 def export_data(df, filename, format='csv', **kwargs):
283     """Custom function to export data in different formats"""
284     if format == 'csv':
285         df.to_csv(filename, index=False, **kwargs)
286     elif format == 'excel':
287         df.to_excel(filename, index=False, **kwargs)
288     elif format == 'json':
289         df.to_json(filename, orient='records', indent=2, **kwargs)
290     elif format == 'parquet':
291         df.to_parquet(filename, index=False, **kwargs)
292     else:
293         raise ValueError(f"Unsupported format: {format}")
294
295     print(f"Data exported to {filename} in {format} format")
296
297     # Use custom export function
298     export_data(export_df, 'custom_export.csv', format='csv')
299     export_data(export_df, 'custom_export.json', format='json')
300
301     # 13. Performance Comparison
302     print(f"\n--- 13. Performance Comparison ---")
303     import time
304
305     # Time different export formats
306     formats = ['csv', 'excel', 'json', 'parquet', 'pickle', 'hdf5', 'feather']
307     export_times = {}
308
309     for format_type in formats:
310         start_time = time.time()
311         if format_type == 'csv':
312             export_df.to_csv(f'perf_test.{format_type}', index=False)
313         elif format_type == 'excel':
314             export_df.to_excel(f'perf_test.{format_type}', index=False)
315         elif format_type == 'json':
316             export_df.to_json(f'perf_test.{format_type}', orient='records')
317         elif format_type == 'parquet':
318             export_df.to_parquet(f'perf_test.{format_type}', index=False)
319         elif format_type == 'pickle':
320             export_df.to_pickle(f'perf_test.{format_type}')
321         elif format_type == 'hdf5':
322             export_df.to_hdf(f'perf_test.{format_type}', key='data')
323         elif format_type == 'feather':
324             export_df.to_feather(f'perf_test.{format_type}')
325
326         end_time = time.time()
327         export_times[format_type] = end_time - start_time
328         print(f"{format_type.upper()} export time: {export_times[format_type]}")
329
330     # Time different import formats
331     import_times = {}
332
333     for format_type in formats:
334         start_time = time.time()
335         if format_type == 'csv':
336             pd.read_csv(f'perf_test.{format_type}')
337         elif format_type == 'excel':
338             pd.read_excel(f'perf_test.{format_type}')
339         elif format_type == 'json':
340             pd.read_json(f'perf_test.{format_type}')
341         elif format_type == 'parquet':
342             pd.read_parquet(f'perf_test.{format_type}')
343

```

```

346         pd.read_parquet(f'perf_test.{format_type}')
347     elif format_type == 'pickle':
348         pd.read_pickle(f'perf_test.{format_type}')
349     elif format_type == 'hdf5':
350         pd.read_hdf(f'perf_test.{format_type}', key='data')
351     elif format_type == 'feather':
352         pd.read_feather(f'perf_test.{format_type}')
353
354     end_time = time.time()
355     import_times[format_type] = end_time - start_time
356     print(f"{format_type.upper()} import time: {import_times[format_type]}")
357
358 # 14. File Size Comparison
359 print(f"\n--- 14. File Size Comparison ---")
360
361 file_sizes = {}
362 for format_type in formats:
363     filename = f'perf_test.{format_type}'
364     if os.path.exists(filename):
365         file_size = os.path.getsize(filename)
366         file_sizes[format_type] = file_size
367         print(f"{format_type.upper()} file size: {file_size / 1024} MB")
368
369 # 15. Data Type Preservation
370 print(f"\n--- 15. Data Type Preservation ---")
371
372 # Check data types after export/import
373 original_dtypes = export_df.dtypes
374 print(f"Original data types:\n{original_dtypes}")
375
376 # Check CSV data types
377 csv_dtypes = imported_csv.dtypes
378 print(f"CSV data types:\n{csv_dtypes}")
379
380 # Check Parquet data types
381 parquet_dtypes = imported_parquet.dtypes
382 print(f"Parquet data types:\n{parquet_dtypes}")
383
384 # Check Pickle data types
385 pickle_dtypes = imported_pickle.dtypes
386 print(f"Pickle data types:\n{pickle_dtypes}")
387
388 # 16. Error Handling
389 print(f"\n--- 16. Error Handling ---")
390
391 # Handle missing files
392 try:
393     missing_data = pd.read_csv('nonexistent_file.csv')
394 except FileNotFoundError:
395     print("File not found error handled gracefully")
396
397 # Handle corrupted files
398 try:
399     # Create a corrupted CSV file
400     with open('corrupted.csv', 'w') as f:
401         f.write("Name,Age\nJohn,25\nJane,30\nCorrupted,line\n")
402
403     corrupted_data = pd.read_csv('corrupted.csv')
404     print("Corrupted file handled gracefully")
405 except Exception as e:

```

```

410         print(f"Error handling corrupted file: {e}")
411
412     # 17. Cleanup
413     print(f"\n--- 17. Cleanup ---")

```

  

```

# Remove test files
test_files = [
    'sample_data.csv', 'sample_data_semicolon.csv', 'sample_data_
    'sample_data.xlsx', 'sample_data_multi.xlsx', 'sample_data.js
    'sample_data_index.json', 'sample_data_values.json', 'sample_
    'sample_data.parquet', 'sample_data.pkl', 'sample_data.db',
    'sample_data.h5', 'sample_data_multi.h5', 'sample_data.feather
    'sample_data.dta', 'custom_export.csv', 'custom_export.json',
    'corrupted.csv'
]

for file in test_files:
    if os.path.exists(file):
        os.remove(file)
        print(f"Removed: {file}")

# Remove performance test files
for format_type in formats:
    filename = f'perf_test.{format_type}'
    if os.path.exists(filename):
        os.remove(filename)
        print(f"Removed: {filename}")

print("\nExporting and importing data completed successfully!")

```

## Summary

This module covered all 11 advanced topics in Pandas DataFrame operations:

1. **GroupBy Operations and Aggregation** - Grouping data and applying various aggregation functions
2. **Pivot Tables and Cross-tabulation** - Creating pivot tables and cross-tabulations for data analysis
3. **Merging and Joining DataFrames** - Combining DataFrames using different join operations
4. **Concatenating DataFrames** - Combining DataFrames vertically and horizontally
5. **Reshaping Data (melt, pivot, stack, unstack)** - Transforming data between wide and long formats
6. **Time Series Operations** - Working with time-based data including resampling and shifting
7. **Window Functions and Rolling Operations** - Applying rolling and expanding operations
8. **Advanced Filtering and Querying** - Complex filtering using boolean indexing and query methods
9. **Multi-level Indexing** - Working with hierarchical indexes for complex data structures

10. **Performance Optimization Techniques** - Best practices for efficient pandas operations

11. **Exporting and Importing Data** - Saving and loading data in various formats

Each topic included comprehensive examples, performance comparisons, and best practices to help you master advanced pandas operations for data analysis and manipulation.